

UNIVERSIDAD NACIONAL DE EDUCACIÓN

Enrique Guzmán y Valle

Alma Máter del Magisterio Nacional

FACULTAD DE CIENCIAS

Escuela Profesional de Matemática e Informática



LENGUAJE DE PROGRAMACIÓN II

Programación orientada a objetos. Objetos. Clases. Herencia. Polimorfismo. Lenguajes. Utilidades para el desarrollo y prueba de programas. Compiladores. Intérpretes. Depuradores. Prueba de programas en lenguaje de programación C o Java.

Examen de Suficiencia Profesional Res. N° 1623-2021-D-FAC

Presentada por:

Lilian Georgette Aviles Gutierrez

Para optar al Título Profesional de Licenciado en Educación

Especialidad: Matemática e Informática

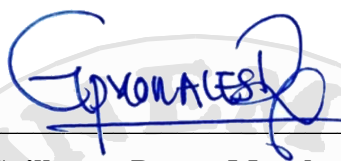
Lima, Perú

2021

LENGUAJE DE PROGRAMACIÓN II

Programación orientada a objetos. Objetos. Clases. Herencia. Polimorfismo. Lenguajes. Utilidades para el desarrollo y prueba de programas. Compiladores. Intérpretes. Depuradores. Prueba de programas en lenguaje de programación C o Java.

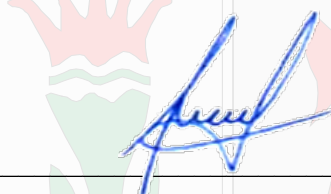
Designación de Jurado Resolución N° 1623-2021-D-FAC



Dr. Guillermo Pastor Morales Romero
Presidente



Dr. Juan Carlos Huamán Hurtado
Secretario



Dra. Jady Luz Vargas Tumaya
Vocal

Línea de investigación: Teorías y paradigmas educativos

UNIVERSIDAD NACIONAL DE EDUCACIÓN
ENRIQUE GUZMÁN Y VALLE
Alma Mater del Magisterio Nacional



FACULTAD DE CIENCIAS
DIRECCIÓN DE UNIDAD DE INVESTIGACIÓN

“AÑO DEL FORTALECIMIENTO DE LA SOBERANÍA NACIONAL”

INFORME Nº067– 2022 – DUI

Asunto : APROBACIÓN DE ORIGINALIDAD DE MONOGRAFIA

Fecha : La Cantuta, 19 de mayo del 2022

Yo, Director de la Unidad de Investigación de la Facultad de Ciencias, he aplicado el PROGRAMA TURNITIN a la Monografía titulada:

LENGUAJE DE PROGRAMACIÓN II

Programación orientada a objetos. Objetos. Clases. Herencia. Polimorfismo. Lenguajes. Utilidades para el desarrollo y prueba de programas. Compiladores. Intérpretes. Depuradores. Prueba de programas en lenguaje de programación C ó Java.

Sustentación de la Monografía, Resolución No 1623-2021-D-FAC

AVILES GUTIERREZ LILIAN GEORGETTE

Código No 20151487

La citada monografía, tiene un índice de **SIMILITUD** del **7%**, verificable en el reporte de originalidad del programa Turnitin; grado de coincidencia mínimo que convierte al trabajo académico en **ACEPTABLE** y no constituye plagio.

Por lo tanto, cumple con las normas del estilo APA, establecidas por la Universidad Nacional de Educación Enrique Guzmán y Valle.

Atentamente.



Dr. Enzo Carol Foy Valencia
Unidad de Investigación – FAC
Director

c.c.
Rocío T.

Unidad de Repositorio Institucional y Biblioteca Virtual

LENGUAJE DE PROGRAMACIÓN II

Examen de Suficiencia Profesional Res. N° 1623-2021-D-FAC

Aviles Gutierrez, Lilian Georgette

Especialidad: Matemática e Informática

13 gestion.pe <1 % >

Nro. Registro RIDUNE: 2022 - FAC - 065

Nro. Registro RIDUNE: 2022 - FAC - 065

Dedicatoria

Este trabajo se lo dedico a mis padres, que con mucho esfuerzo me apoyaron en mis estudios; les agradezco infinitamente mi camino de formación profesional, en especial a mi padre Gilmer Aviles Huatuco, que desde el cielo guía mis pasos, los recuerdos y enseñanzas que tengo de él me permiten continuar.

Además, se lo dedico a mis maestros, por sus enseñanzas brindadas, ya que me orientaron y educaron con sabiduría.

Índice de contenidos

Portada.....	i
Hoja de firmas de jurado.....	ii
Dedicatoria	iii
Índice de contenidos	iv
Lista de figuras	vii
Introducción.....	viii
Capítulo I. Lenguaje de programación	9
1.1 Lenguaje de programación	9
1.1.1 Paradigmas en lenguaje de programación.....	11
1.1.2 Tipos de paradigmas de programación.	12
1.1.2.1 Paradigma imperativo.	12
1.1.2.2 Paradigma heurístico.....	13
1.1.2.3 Paradigma funcional.	14
1.1.2.4 Paradigma lógico.	16
1.1.2.5 Paradigma basado en objetos.	18
1.2 Elementos de programación	20
1.3 Razones de estudio del lenguaje de programación.....	26
1.4 Evolución de los lenguajes de programación	27
1.5 Tipos de lenguajes de programación	29
1.5.1 Lenguaje de programación más usados.....	30
Capítulo II. Programación orientada a objetos OOP	33
2.1 Programación orientada a objetos.....	33
2.2 Distinción de objetos y clases.....	34
2.2.1 Objetos.	34

2.2.2 Clases.	35
2.2.3 Mensajes.....	35
2.3 Principios.....	36
2.3.1 La herencia.	36
2.3.1.1 Tipos de herencia en Java.	37
2.3.2 Polimorfismo.....	38
2.3.3 Abstracción.....	40
2.3.4 Encapsulamiento.	42
Capítulo III. Utilidades para el desarrollo y prueba de programas	45
3.1 Compiladores.....	45
3.1.1 Primer compilador.....	46
3.1.2 Intérpretes.....	47
3.2 Depuradores.....	47
3.3 Prueba de programas en lenguaje Java.....	51
3.3.1 Concepto.....	51
3.4 Tipo de pruebas	52
3.4.1 Pruebas unitarias.	52
3.4.2 Pruebas de integración.	53
3.4.3 Pruebas de sistemas.....	55
3.4.4 Pruebas de validación.....	56
3.5 Ventajas	57
Aplicación didáctica	58
Síntesis.....	72
Apreciación crítica y sugerencias	73
Referencias	74

Apéndice.....	78
---------------	----

Lista de figuras

Figura 1. Lenguajes de programación	11
Figura 2. Lenguajes con enfoque imperativo	13
Figura 3. Ejemplo de definición de funciones en el paradigma funcional	15
Figura 4. Lenguajes en el paradigma funcional.....	16
Figura 5. Programacion en código en el paradigma funcional.....	17
Figura 6. Lenguajes en el paradigma lógico.....	17
Figura 7. Programación de clases y herencia	19
Figura 8. Lenguajes orientados a objetos	20
Figura 9. Diagrama de flujo.....	24
Figura 10. Símbolos gráficos.....	25
Figura 11. Seudocódigos gráfico	25
Figura 12. Primera forma del lenguaje de programacion	28
Figura 13. Programación orientada a objetos	34
Figura 14. Atributos del objeto.....	35
Figura 15. Tipos de herencia en Java	38
Figura 16. Ejemplo de polimorfismo.....	39
Figura 17. Ejemplo de polimorfismo en ejecucion.....	39
Figura 18. Ejemplo de abstracción	41
Figura 19. Encapsulamiento	43
Figura 20. Proceso de compilador	46
Figura 21. Proceso de depuración.....	48
Figura 22. Integración incremental.....	54

Introducción

Actualmente vivimos en un mundo globalizado en donde el acceso a las nuevas tecnologías en diversas instituciones educativas se hace más comunes, cambiando la manera en cómo los usuarios se conectan a la red, especialmente los más jóvenes, quienes están tan familiarizados con los diversos aparatos electrónicos como son las tabletas y teléfonos inteligentes, así como la conexión a internet mediante dichos dispositivos, lo que ha ido expandiendo su uso alrededor del mundo. En el mundo actual, para cualquier entidad le es necesaria tener presencia en la red y el uso de mecanismos que mejoren su eficiencia. En el caso específico de la educación, esta no se encuentra ajena a esta dinámica tecnológica, evidenciándose que desde cada área tienden a proporcionar herramientas que faciliten la optimización de los procesos de enseñanza y aprendizaje, como por ejemplo el curso de matemática, que en este caso se enmarca en el uso de la programación Java para la enseñanza de números enteros. Es por ello que, al ser un tipo de lenguaje de programación, es posible constituir como un lenguaje orientado a los objetos, puesto que en la actualidad es necesario que los contenidos sean además accesibles y entendibles por los estudiantes. Por tanto, es imperativo para este mundo actual y teniendo en cuenta que la forma de aprendizaje de los estudiantes en la actualidad es más significativa cuando va acompañado de un lenguaje visual, de tal forma que puedan pasar del nivel abstracto como a las matemáticas lo concreto y de lo concreto nuevamente a lo abstracto. En la vida actual no basta solo con disponer con recursos tecnológicos para la educación, sino también saber innovar su uso y ello represente un avance y un aporte a las ciencias de la educación.

Capítulo I

Lenguaje de programación

1.1 Lenguaje de programación

Dicha terminología se basa en un sistema de comunicación estructurado, compuesto por una serie de simbologías, palabras y reglas, que pueden realizar el entendimiento entre programadores y computadoras. Estas son la serie de orientaciones y algoritmos brindadas por el desarrollador, los cuales ayudan a controlar la actuación física y lógica de un dispositivo, con la finalidad de señalar la índole de los datos a intervenir y las labores que deberá realizar en un espectro de circunstancias (Hirsh, 2007).

La programación es entendida por realizar la codificación del algoritmo, el cual se basará en determinado programa. A dicho algoritmo, al ser escrito en lenguajes de programación se le identifica como código fuente (Joyanes, 2008).

Los mismos serán validados e interpretados por un proceso denominado compilación, para que la computadora logre decodificarlos. El lenguaje informático tiende a complejizarse, pero paradójicamente busca ser entendible y en algunos casos intuitivos para un uso más eficaz. Realizar el descifrado de estas instrucciones se torna extremadamente complejo para las personas (Durango, Arias y Gracia, 2016).

A lo que se refiere a los lenguajes de bajo nivel o de ensamblador, incluyen los procedimientos en simbología nemotécnica, las cuales se componen de palabras en el idioma inglés para facilitar el desarrollo de los softwares, estos son el reemplazo de los procedimientos en lenguaje binario (Joyanes, 2008).

Al referirse al mismo concepto, pero de gran nivel, se comprende como las indicaciones en lengua humana, el mismo se realiza con el fin de que los ordenadores realicen actos y operaciones particulares en conjunción a un determinado algoritmo o pseudocódigo, ello debido a que los ordenadores no les es posible reconocer estas ordenas escritas de forma humana, para ello entran en escena los programas denominados como compiladores, los cuales juegan el vital papel de traducir estas escrituras a uno, el cual pueda ser reconocido por las máquinas.

La sociedad, en sus diversos ámbitos, se conecta actualmente al internet, esto la ha transformado drásticamente en los tiempos contemporáneos. Siendo esto contribuido en gran medida a las enormes cuotas que el mercado está accediendo a través del uso de los denominados smartphone y tablet, por la conectividad que se deriva de dichos aparatos, además de ser una de las principales virtudes que encaminan que sean usados. Por ello se han bautizado estos aparatos como unas de las formas fundamentales de conectarse a la web, siendo entonces una alternativa viable y sencilla antes métodos anteriores.

Durante mucho tiempo la presencia del internet fue fundamental para los centros educativos, pero en los días actuales ya se tornan insuficientes para los escenarios debido a que las realidades han cambiado, puesto que se ha tornado imperante que el uso de múltiples contenidos pueda realizarse en todos los terminales móviles actuales y no solo se acceda mediante computadores de escritorio.

Conociendo esto, es preponderante la necesidad de brindar una web que se adapte para ser usada y disfrutada en los teléfonos inteligentes, además conllevando aplicaciones

individualizadas para marcar la diferencia con el pasado y se conviertan en herramientas útiles para la pedagogía (Fernández, 2016).

1.1.1 Paradigmas en lenguaje de programación.

A través de la evolución histórica de programar han germinado múltiples técnicas para hacerlo, estas se han adecuando a las necesidades técnicas y de las tecnologías de la información para su contemporaneidad. Si bien el desarrollo de los software ha presentado contrastes entre los mismos, la finalidad siempre se ha compartido, siendo proporcionar facilidades para crear y mantener los programas informáticos. Dichas metodologías han mutado como un conjunto de saberes en programación, acuñados como paradigmas de programación.

Estos paradigmas se entienden como un conjunto de metodologías, formas de organizar y estructura de los procesos que se debe llevar a cabo por un programa.

Al proceso de indagación de los lenguajes de programación, en lo que respecta a los enfoques que estos tienen, se le denomina paradigmas de programación, aludiendo a la perspectiva que puede tener el que diseña y el que programa en estos lenguajes.

Los aspectos antes mencionados existen como múltiples, como los imperativos o por procedimientos, como por ejemplo PASCAL, el lenguaje C o el JAVA, C++, el cual se orienta a objetos (Joyanes, 2008).



Figura 1. Lenguajes de programación. Fuente: Autoría propia.

1.1.2 Tipos de paradigmas de programación.

Estos pueden ser clasificados por autores como los basados en objetos, lógicos, funcional, heurístico e imperativo, los cuales se desarrollan (Rodríguez, 2016).

Bajo este enfoque se tiene cuatro paradigmas, los cuales son:

- Paradigma por procedimientos o paradigma imperativo
- Paradigma declarativo
- Paradigma funcional
- Paradigma orientado a objetos

1.1.2.1 Paradigma imperativo.

Estos contribuyen con cálculos cambiando el estado y entienden el estado de una memoria como un estado. Dichos lenguajes se estructuran en bloques, refiriéndose a ámbitos anidados, estos se pueden anidar adentro de otros bloques y conteniendo sus variables particulares (Rodríguez, 2016).

Al escribir un programa desarrollan un lenguaje funcional que consta solo de actividades particulares, es decir, no a modo de subrutinas clásicas del lenguaje imperativo, más bien como funciones meramente aritméticas, en las cuales se comprobarán propiedades como la transparencia referencial, la cual indica el significado de la expresión, dependen de manera única del significado de la subexpresión; por ende, la ausencia absoluta de efectos secundarios. Su finalidad en alcanzar lenguajes expresivos y aritméticamente entendibles, donde no deba disminuirse los niveles de la máquina para realizar la descripción los procesos que se ejecutan por el programa (Inca, 2012).

Las secuencias de cálculos que realiza los programas se registrarán sola y únicamente por la reescritura de definiciones más extensas a otras, las cuales sean más puntuales y

precisas, utilizando las denominadas definiciones directas. Estos paradigmas para programar de forma frecuente son conocidos como algorítmico (Sierra y Bates, 2005).

Otro rasgo distintivo de dichos lenguajes es la inexistencia de asignaciones variables y la ausencia de construcciones estructuradas, tal como las secuencias o iteraciones, las cuales en la faena obligan que las repeticiones de estas orientaciones deban ser realizadas mediante funciones recursivas.

Algunos de los lenguajes imperativos son:

- C, C++, C#, Basic, Java, Perl.



Figura 2. Lenguajes con enfoque imperativo. Fuente: Autoría propia.

1.1.2.2 Paradigma heurístico.

Se presenta como una modalidad de resolver problemas en la que se integra una técnica heurística para su solución basadas en una reconstrucción más adecuada de la estructura del problema (Rodríguez, 2016).

Esta se puede entender como la forma de programar en informática que utiliza buenas reglas lógicas (reglas generales) para resolver problemas que proporcionan uno de los varios enfoques que parecen más prometedores pero que no necesariamente garantizan la eficacia (Inca, 2012).

Programar de forma heurística involucra una forma de modelar el problema con respecto a cómo se presenta la estructura, destrezas de búsqueda y metodologías de resolución, los cuales constituyen dicho paradigma.

Esta forma de programar encuentra aplicación de manera más intensa en el ámbito de la Inteligencia Artificial (IA), además sobre la Ingeniería del Conocimiento, incorporándola recientemente en el ámbito académico (Trejos, 2013).

La programación se puede presentar y ser utilizada en diferentes ámbitos como:

- Técnicas para las investigaciones en la consecución de objetivos en problemas no algorítmicos, o con algoritmos, los cuales generen explosiones combinatorias.
- Como métodos de aproximación para resolver problemas usando funciones de evaluación heurísticas.
- Como métodos de acotación para reproducir estrategias de programas, a pesar de que ellos no son propiamente heurísticos.

1.1.2.3 Paradigma funcional.

Su génesis procede del cálculo lambda, la cual es una teoría aritmética desarrollada por Alonzo Church en soporte de las investigaciones de predictibilidad. Este, en términos generales, es un método para realizar ciertas construcciones sencillas de expresar o leer provenientes del cálculo mencionado, el cual se fundamenta en el modelo aritmético de composición funcional. Dicho modelo es lo resultante de un cálculo en la entrada del sucesivo y así continuamente hasta que una composición resulte en los valores deseados (Álvarez, Ranilla, Unzueta y González, 2015).

Es inexistente la concepción de celda de memoria asignada o modificada. En cambio, pueden encontrarse valores medios, los cuales resultan de cálculos anteriores y entradas para cálculos posteriores. Además, son inexistentes las sentencias obligatorias y

las funciones que presentan transparencia referencial. Este tipo de programación implica los conceptos de función como un objeto principal, lo que deviene que las funciones podrían pasarse como parámetros, calcularse y devolverse valores normales, mezclándose con otros tipos de datos cuando sean calculados (Álvarez *et al.*, 2015).

Para dicho paradigma, el programador deberá comprender las soluciones como una serie de composiciones de funciones. Las formas en las cuales se pueden especificar estas funciones pudiesen ser variables, por definición estas podrán ser especificadas de forma procedimental o matemática sin secuencias de control (Joyanes, 2008).

Dentro de las características de este paradigma, tenemos:

- La computación se realiza mediante la evaluación de expresiones.
- Definición de funciones
- Funciones como datos primitivos
- Valores sin efectos laterales, no existe la asignación • Programación declarativa
- Lenguajes: LISP, Scheme, Haskell, Scala

```
(define (factorial x)
  (if (= x 0)
      1
      (* x (factorial (- x 1)))))

(factorial 8)
40320
(factorial 30)
265252859812191058636308480000000
```

Figura 3. Ejemplo de definición de funciones en el paradigma funcional. Fuente: Autoría propia.



Figura 4. Lenguajes en el paradigma funcional. Fuente: Autoría propia.

1.1.2.4 Paradigma lógico.

La forma de programar como paradigma se basa en la lógica. Los softwares escritos en este lenguaje se crean de forma única mediante expresiones lógicas; por lo tanto, van del verdadero o falso, a diferencia de expresiones de preguntas o expresión imperativa, como ejemplificación de este lenguaje esta Prolog, acrónimo de programación lógica, el cual cobra importante participación en las investigaciones de inteligencia artificial. Este es bastante disímil a los imperativos tal como C, Pascal, Fortran, entre otros, es alejado de los funcionales tal como Lisp. Estos últimos mencionados, los comandos, se suelen ejecutar de forma secuencial, por ende, continuando a otro, en la misma secuencia que se encuentran escritos, estos sufren variaciones al alcanzar una instrucción de control (Rodríguez, 2016).

- Definición de reglas

- Unificación como elemento de computación
- Programación declarativa
- Lenguajes: Prolog, Mercury, Oz

```

padrede('juan', 'maria'). % juan es padre de maria
padrede('pablo', 'juan'). % pablo es padre de juan
padrede('pablo', 'marcela').
padrede('carlos', 'debora').

hijode(A,B) :- padrede(B,A).
abuelode(A,B) :- padrede(A,C), padrede(C,B).
hermanode(A,B) :- padrede(C,A), padrede(C,B), A \== B.

familiarde(A,B) :- padrede(A,B).
familiarde(A,B) :- hijode(A,B).
familiarde(A,B) :- hermanode(A,B).

?- hermanode('juan', 'marcela').
yes
?- hermanode('carlos', 'juan').
no
?- abuelode('pablo', 'maria').
yes
?- abuelode('maria', 'pablo').
no

```

Figura 5. Programación en código en el paradigma funcional. Fuente: Autoría propia.

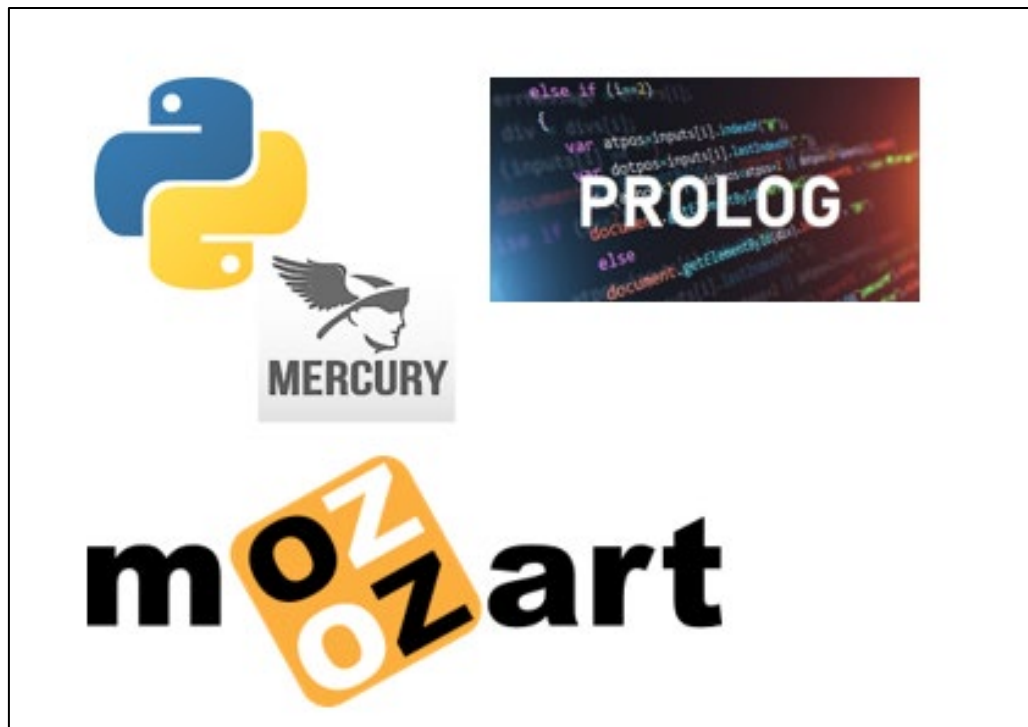


Figura 6. Lenguajes en el paradigma lógico. Fuente: Autoría propia.

1.1.2.5 Paradigma basado en objetos.

Esta programación encauzada a objetos (OOP) es una forma de programar, la cual se ha extendido desde el período de los 80. Este es un paradigma de programar que puntualiza los software en términos de clases de objetos, estos siendo unidades de estado, entendiéndose como datos, comportamiento, entendiéndose como los procedimientos e identidad, comprendiéndose como las propiedades de dicho objeto que lo diferencia de los otros. Esta forma de programar expresa un programa como un compendio de dichos objetos, los cuales se asisten entre ellos para efectuar los cometidos. Esto facilita la escritura, el mantenimiento y la reutilización de programas y módulos (Sierra y Bates, 2005).

De este modo, un objeto incluye la totalidad de la información que permite definirla e identificarla con respecto a otros que pertenecen a distintas clases y también entre sus mismas clases, ya que pueden ser valores bastantes diferentes en cuanto a los atributos. Tiene a la vez mecanismos de cooperación que benefician las comunicaciones entre objetos de las mismas clases o diferentes y en derivación, la modificación en sí de dichos objetos. Estas particularidades conllevan a ser tratados como unidades inseparables, es decir, los datos de estos y procesos no se encuentran aislados. Dadas estas propiedades de conjunto, al poseer un conjunto de atributos, se precisan métodos para su manejo, lo que hace que de manera concluyente estos conceptos se encuentren ligados de manera estrecha, entendiendo la persona que programa el énfasis que debe otorgarles para no desligarlos o resaltar atributos a favor de los procedimientos o lo contrario (Trejos, 2013).

Los fundamental que separa este paradigma de la programación imperativa es:

- Este paradigma es más actual, siendo la evolución de la forma de programar imperativa, que incorpora conceptos que existían anteriormente con algunos nuevos en favor de diseños de una ramificación de lenguajes.
- Está fundamentado en lenguajes, los cuales toleran sintáctica y semánticamente la adición de datos de tipos abstractos y operaciones.
- Incorpora mecánicas como el polimorfismo y la comunicación por mensajes entre objetos a su ambiente en que se ejecutan.

Los lenguajes más conocidos de este tipo son C++, .NET, Ada, Clarion, C#, Delphi, Java, Eiffel y Lexico (Sierra y Bates, 2005).

Dentro de las características de este paradigma, tenemos:

- Definición de clases y herencia
- Objetos como abstracción de datos y procedimientos
- Polimorfismo y chequeo de tipos en tiempo de ejecución
- Ejemplo en Java, Visual Basic

```
public class Bicicleta {  
    public int marcha;  
    public int velocidad;  
  
    public Bicicleta(int velocidadInicial, int marchaInicial) {  
        marcha = marchaInicial;  
        velocidad = velocidadInicial;  
    }  
  
    public void setMarcha(int nuevoValor) {  
        marcha = nuevoValor;  
    }  
  
    public void frenar(int decremento) {  
        velocidad -= decremento;  
    }  
  
    public void acelerar(int incremento) {  
        velocidad += incremento;  
    }  
}
```

Figura 7. Programación de clases y herencia. Fuente: Autoría propia.



Figura 8. Lenguajes orientados a objetos. Fuente: Autoría propia.

1.2 Elementos de programación

Entre los elementos para programar más frecuentados para lograr buenas prácticas de programación se encuentran, por muestra, los algoritmos, pseudocódigos, diagrama de flujos, diagrama de bloques, mapa de relación de entidades y diagrama de flujo de datos (Moreno, 2016).

Algoritmos: Es una secuencia finita de medidas que definen unas secuencias de operaciones a efectuar para solucionar un determinado problema o una determinada clase de problema; por ende, es un método para solventar el problema. Es una serie de operaciones sistemáticas en una forma que se pueda solucionar un problema, es reducido el número de personas que conocen que la terminología proviene de un catedrático de la astronomía, la geografía y matemático de origen mahometano Al Jwarizmi (Mancilla, Ebratt y Camacho, 2016).

La rama científica encargada del estudio de la misma es denominada algorítmica, yaciendo su génesis en la afamada Máquina de Turing, la cual hizo formal estos conceptos

en un modelo computarizado. Este concepto posee una relación similar con las funciones aritméticas, es decir, toman una entrada y ocasionan una salida para ello y ser considerado como tal, deben poseer las características de ser eficientes, finitas y definidas (Márquez, 2017).

La propiedad, se tiene que los mismos se expresan en términos de un:

- Lenguaje común
- Diagramas de flujo
- Pseudocódigo
- Lenguajes de programación

Cabe señalar que, para una mejor comprensión, los algoritmos deben expresarse gráficamente, este tipo de gráfico se denomina diagrama de flujo de algoritmos.

Los algoritmos son de naturaleza general y pueden ser aplicadas a todas las operaciones o problemas matemáticos. Formular los algoritmos fue una gran evolución para las matemáticas como ciencia, ya que podía resolver un número infinito de problemas. Para lograr esto deberán cumplir con ciertas particularidades, la principal de estas es que el desarrollo a proseguir deberá ser descrito con precisión, cada operación deberá ser exacta y general, puede ser aplicada a la totalidad de elementos de la misma clase (Mancilla *et al.*, 2016).

El algoritmo por sí mismo no es capaz de resolver problemas, son resueltos debido al producto del ejercicio de las instrucciones dictadas por este. Es válido mencionar, desde el planteamiento en sí, que la realización de estos es originaria e innato en la especie humana y en la generalidad de las situaciones es de forma no consciente, es decir, los individuos suelen solucionar problemas sin acudir a aplicar de un determinado algoritmo; sin embargo, cuando se encuentran problemas de mayor grado de desafío es imperioso parar, cavilar y buscar en solucionarlo (Moreno, 2016).

Para programar se hace uso, en formas de oraciones, en algunos lenguajes de la programación, de esta forma el modo de escribirlos dependerá de los lenguajes que se estén usando para programar. Los algoritmos también pueden ser representados de manera gráfica mediante el diagrama de flujos o formas análogas, con la finalidad de que sean comprendidas de forma fácil, sobre todo para los que no están inmersos en el mundo de la programación, haciéndolos más universalizados, puesto que no se enmarcan en lenguajes de programación particulares (Beristain, 2014).

El algoritmo podrá ser escrito de igual forma en pseudocódigo, lo que conlleva un fácil entendimiento de igual forma. Las computadoras intentan interpretar y ejecutar diagrama de flujos y pseudocódigo, pero no consiguen la maleabilidad, el poder y la rapidez de los algoritmos escritos completamente en cualquier lenguaje para programar. Estos también se podrán expresar en lenguaje natural, pero ello podría dar lugar a ambigüedades y malas interpretaciones (Márquez, 2017).

Los algoritmos en programación, un software de ordenador, son los algoritmos los que le dictan a la máquina las pautas específicas para efectuar una labor. Estos están estrictamente determinados para que el computador alcance descifrarlos.

La disposición ordenada en la cual se ejecuta los pasos que conforman un algoritmo es cardinal, esta forma ordenada será de arriba para abajo, ejecutando las instrucciones consecutivamente de un código, podrá ser diferente el flujo o el orden de ejecución de los pasos según los parámetros iniciales o que sean ingresados mientras se ejecuta; estos flujos serán gestionados por las estructuras de control (Manrique, 2020).

Algoritmos predefinidos, hay la existencia definida de los mentados, los cuales son altamente eficaces aritméticamente, como por ejemplo los de búsqueda, generalmente son usados por las personas que programan para darles uso en sus códigos personales (Rojas, 2018).

Un gran número de sistemas especializados hacen uso de algoritmos para razonar, pero conllevan una limitante fundamental, la cual se basa en que podrá solucionar las situaciones puntuales anteriores para las cuales fue creada por el individuo.

Hay diferentes tipos que permiten razonar, como los algoritmos estáticos, los cuales no varían su función, a pesar del tipo de problemas que se le presenten.

Hay de la misma forma los probabilísticos, los cuales no hacen uso de valores de verdad y, por último, los de tipo adaptativo, los cuales poseen ciertas capacidades para aprender.

Diagramas de flujo, estos son una forma de exhibir de manera visual los flujos de datos mediante los sistemas de procesamiento de informaciones, estos diagramas exponen en cuáles operaciones y en cuál sucesión se hace necesario la resolución de un problema asignado.

Se puede observar como un organigrama que representa de forma esquemática e ilustrativa la sucesión de operaciones que se debe realizar para alcanzar la resolución; estos se suelen dibujar previo a iniciar la programación de un código informático, los mismos ayudan a comunicar la información entre los profesionales que programan y los empresarios. Estos juegan un rol fundamental al momento de la faena ante un problema y ayudan a comprender problemas especialmente difíciles y dilatados. Posterior a su creación se convierte prácticamente sencillo realizar la escritura del programa en lenguajes de altos niveles, además de que brindan aspectos positivos al momento de dar a conocer el programa a terceros; por ende, es justo acotar que dichos diagramas de flujos son primordiales y se convierten en necesarios para crear un legajo de su complejidad para el futuro (Mancilla *et al.*, 2016).

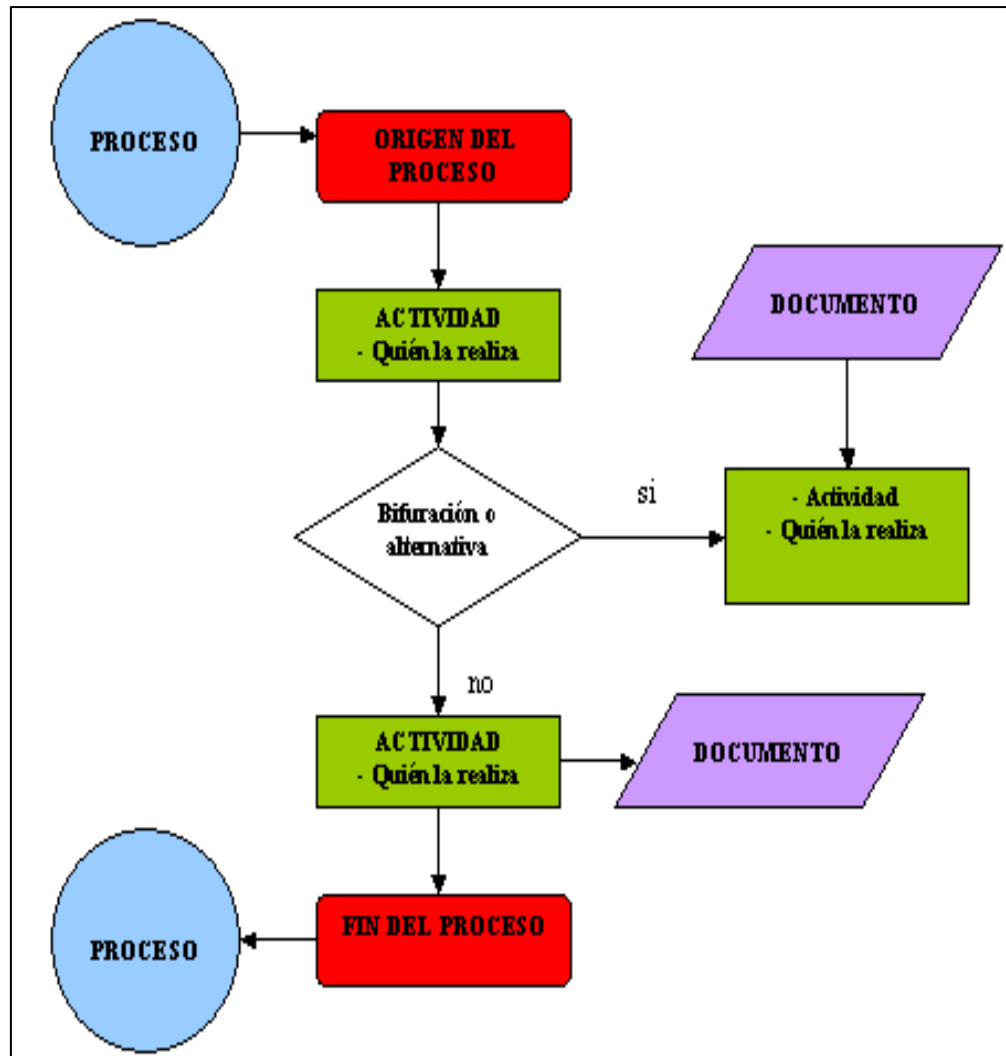


Figura 9. Diagrama de flujo. Fuente: Autoría propia.

Símbolos gráficos: El compendio de simbologías gráficas para crear estos diagramas, se hace uso de manera específica para las operaciones matemáticas y de relación condicional. Estas figuras geométricas, lógicamente estructuradas, nos permiten establecer situaciones condicionales que nos ayudarán a comprender procesos que requieran saber qué pasa en alguna situación y qué pasa estas situaciones de manera tal que nos proporcionará una perspectiva amplia y completa para planificar y diseñar sistemas de información que requieran que se hallan tomar en cuenta todas la situaciones posibles y en ese punto un diseñador de sistemas debe tener estas habilidades.

Por lo tanto, se presenta ejemplificar dichos símbolos usualmente usados:

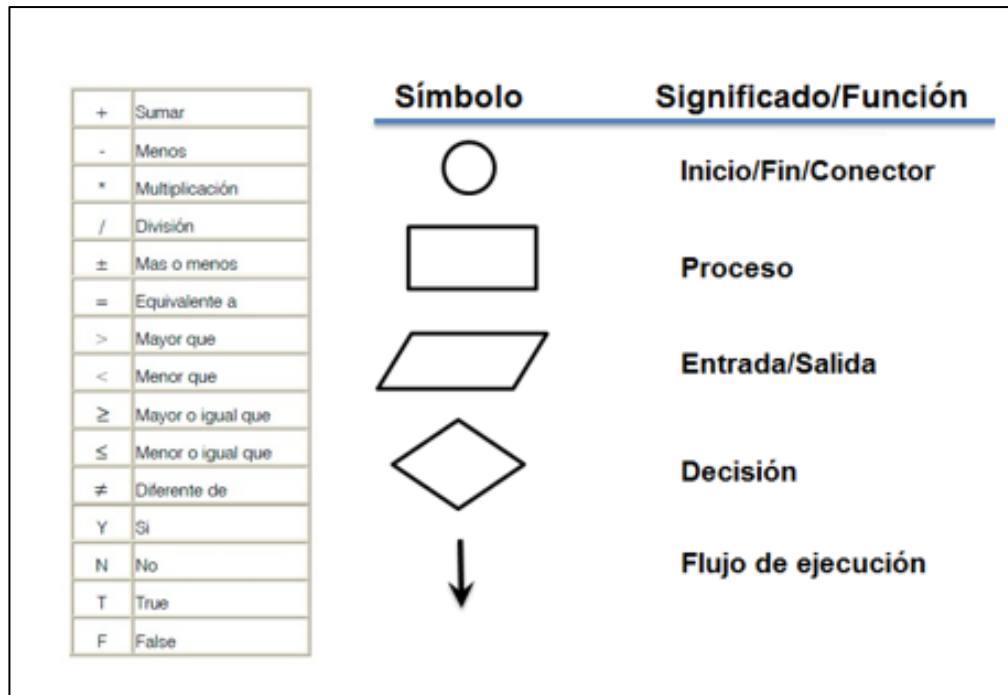


Figura 10. Símbolos gráficos. Fuente: Autoría propia.

Pseudocódigos: El pseudocódigo hace referencia a un lenguaje no natural y no formal, el cual apoya al individuo que programa en el desarrollo de algoritmos. Estos pseudocódigos son herramientas para diseñar detalladamente basándose en textos, algorítmicos. Las reglamentaciones para estos son sensatamente simples. La totalidad de las declaraciones que muestren dependientes deberán estar sangradas, incluyendo swicth, for, do, while, if (Lorente, Cervantes y Salgado, 2020).

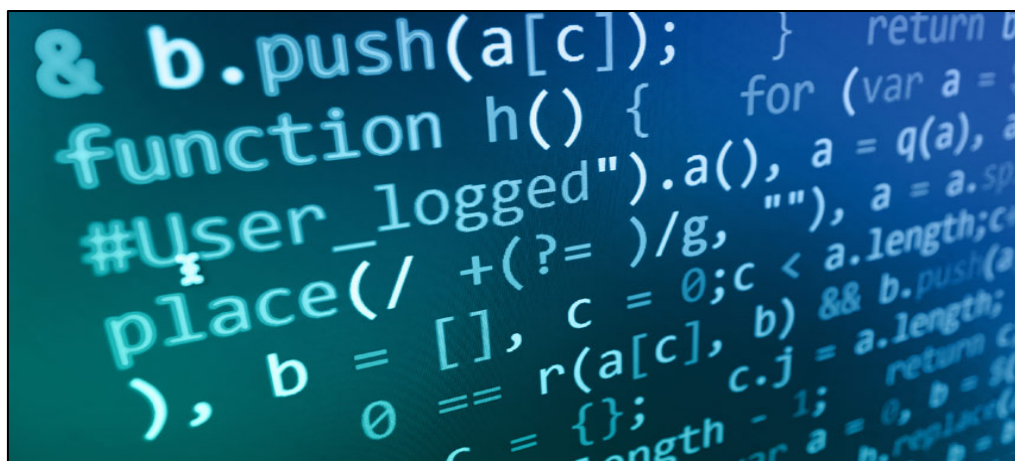


Figura 11. Seudocódigos gráficos. Fuente: Autoría propia.

Entre los beneficios de hacer uso del pseudocódigo está que podrán ser usadas en unas hojas de papel, pueden representarse de manera fácil operaciones iterativas de complejidad, son transportables dado que pueden ser codificadas en cualquier lenguaje para programar, el posible visualizar los distintos niveles de las operaciones de manera despejada (Mancilla *et al.*, 2016).

1.3 Razones de estudio del lenguaje de programación

Estos denotan diferentes tipologías para expresarse y pautas de estructura lógica, las cuales utilizan para concebir tareas repetidas y sistematizadas.

Estas son fundamentales porque conllevan la generación de diferentes sistemas que se utilizan para ocupaciones que cubren las exigencias de los interesados. Hay una múltiple diversidad de dichos lenguajes, los cuales, además, se adaptan a diferentes grados de requisitos; sin embargo, esta pluralidad se puede dividir en dos amplios conjuntos: lenguajes para programar estructurados y los orientados a objetos. Y, básicamente, la totalidad de las cosas que están inmersas en la informática se debe a estos lenguajes (Gortázar, Martínez y Fresno, 2016).

Una computadora le faculta efectuar cálculos a altas velocidades; sin embargo, ello sería imposible de no ser por los softwares ejecutados en las mismas.

Estos se presentan de varios tipos, como los sistemas operativos, siendo quizás el fundamental para los demás, puesto que en él se ejecutan los diversos aplicativos.

Para desarrollarlo persistentemente se necesitan los lenguajes para programar, los cuales poseen particularidades puntuales para poder implementarse. Al mismo tiempo, estos deben ser escritos con uno de nivel inferior, lo que permite una diversidad de respuestas a los requisitos que existen (Manrique, 2020).

Hay dos conjuntos amplios de lenguajes para programar, como se mencionó anteriormente. Los primeros son los estructurados, los cuales constan de una serie de instrucciones y órdenes, como también una serie de pautas para aplicarlas; la lógica proposicional juega el papel primordial en dicho aspecto y la totalidad de las operaciones estarán sujetas a esa relación lógica. Y, como segundo lugar, los programas orientados a objetos, la intención será reflejar diversas circunstancias de la existencia real en el lenguaje. Por tanto, se refiere a objetos, clases, herencia, atributos, entre otros.

Estos lenguajes podrían ser divididos en los interpretados y los compilados; en los primeros tienen que interpretados por un segundo programa y en el otro lenguaje tiene que ser un compilador el que lo traduzca, el cual creará un archivo denominado como ejecutable (Gortázar *et al.*, 2016).

La comprensión de los lenguajes para programar podrá ser fundamental desarrollando habilidades de resolver problemas y automatizando actividades.

Seguramente estas cogniciones, sumadas a saberes básicos de algoritmos, pueden generar diversos horizontes completamente novedosos en cuanto a oportunidades de empleo.

1.4 Evolución de los lenguajes de programación

La primera forma de lenguaje para programar vio su génesis en el año 1801, siendo Joseph Marie Jacquard el padre del telar programable, en donde mediante un conjunto de tarjetas se incorporaban a la máquina y esta interpretaba un código a modo de instructivo y se lograba de forma automática el proceso.

La finalidad fue la creación de diversos patrones en las telas, esto le dio acceso a que las personas que poseyeran el telar y el conjunto de tarjetas con perforaciones pudiese crear patrones sin poseer la cualificación especializada en el área (Inca, 2012).

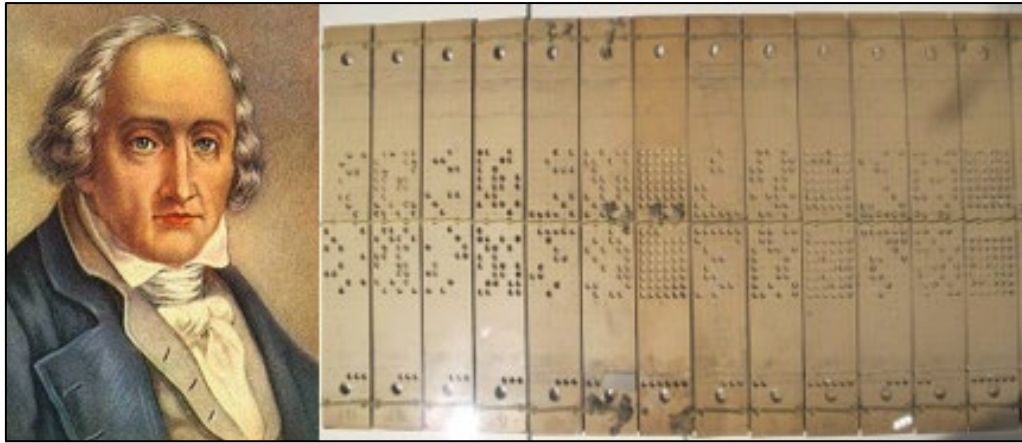


Figura 12. Primera forma del lenguaje de programación. Fuente: Autoría propia.

Se cree que esto pasó a ser el primer lenguaje que permitió programar, puesto que dichas tarjetas fueron usadas posteriormente en las máquinas informáticas novedosas erigidas por Charles Babbage, que dio paso al inicio de los ordenadores (Inca, 2012).

Posteriormente, para los años 1842-1843 aproximadamente, Ada Lovelace se dio a la tarea de traducir los legajos de Luigi Menabrea, el cual se le conoció como matemático en Italia. Ella logró la creación a mano de un lenguaje para realizar el cálculo de, posteriormente conocido, números de Bernoulli con la ayuda del aparato de Babbage (Lázcares y Molina, 2015).

Tiempo después, Alan Turing creó una revolución en materia de computación con una máquina que probaba mediante el uso de algoritmos, se lograba dar solución a los problemas matemáticos de la época, el cual ayudó en el ámbito militar para sus tiempos.

El lenguaje COBOL, en 1959, llegó como inaugural compilador, deviniendo con el apogeo de diversos sistemas para programar, la finalidad fue elaborar un lenguaje universalizado, el cual pudiese ser ejecutado en todos los ordenadores. Para la década de los 60 existían diversos ordenadores, los cuales poseían compatibilidad entre ellos, de esta forma las organizaciones empresariales podrían captar talentos sin acudir a la enseñanza de utilizar compiladores y otros lenguajes, los cuales utilizaban en las organizaciones, estos eran distintos (Lázcares y Molina, 2015).

El Python y Visual Basic, tras años de existencia de diversos compiladores y lenguajes de programación abocados a objetos, nació la web, en dicho momento se engendró la amplia progresión de los lenguajes para programar, innovando con las ideas antiguas. Para el inicio de la década de los 90 nacieron estos lenguajes que han marcado la pauta para las páginas web y HTML, particularizado por el hipertexto, siendo este el código de internet (Luján, 2002).

Java, JavaScript y PHP, estos lenguajes nacieron en 1995, los cuales le han dado forma y determinado a nuestra forma de entender el mundo porque, al fin y al cabo, están presentes en los años 50, en las páginas web y en cuantiosas aplicaciones (Luján, 2002).

C#, con el inicio del siglo XXI, arribaron novedosos lenguajes para programar, tales como Action Script. A pesar de esto, un año después nació C# y Visual Basic .NET; con la suma de los de Java, HTML y el mencionado se le da inicio a los mejores momentos de historia en cuanto al ámbito de programar.

El inicial se ha cimentado como uno de los más sólidos lenguajes, puesto que con él se ha podido realizar infinidad de programas. El segundo se enfoca a objetos que se asocian a Visual Basic, pero siendo profundamente distinto por estar fundado sobre framework .NET de Microsoft (Sánchez, 2016).

Swift, a inicios del 2013 la evolución de los lenguajes para programar se fue ralentizando; sin embargo, han surgido distintos lenguajes de carácter puntual para hacer más ágil programar, como lo es Kotlin y el especificado en este apartado. Instaurado por la compañía Apple, el cual es la piedra angular para programar en iOS (González, 2013).

1.5 Tipos de lenguajes de programación

Observando su propósito y herramientas para las cuales es usado, se podrá definir tres conjuntos de lenguajes para programar:

Lenguaje máquina, siendo este el más rudimentario en cuanto a códigos y está basado en números binarios, es decir, 0 y 1. Esto es usado de manera directa por maquinaria y computadores (Gortázar *et al.*, 2016).

Lenguaje para programar de bajo nivel, siendo más sencillo para su interpretación, pudiendo tener variaciones respecto a las máquinas o computadoras que se vaya a programar.

Lenguaje para programar de alto nivel, dicha clase alberga los comúnmente usados. Se hace uso de términos en inglés, por lo cual ayuda a los programadores al momento de intervenirlos de forma más sencilla que los anteriormente mencionados.

De acuerdo con la evolución generacional, se clasifica a los lenguajes de programación de alto nivel de la siguiente manera:

- 1era. generación, se ve dado entre la máquina y quien la ensambla.
- 2da. generación, se pueden ejemplificar con la llegada de FORTRAN, COBOL.
- 3ra. generación, comienza con los imperativos, los cuales son altamente frecuentados y manipulados aun actualmente como ALGOL 8, PL/I, PASCAL, MODULA.
- 4ta. generación, los usados en la contemporaneidad, generalmente hallados en aplicaciones para la gestión y administración de bases de datos como NATURAL, SQL.
- 5ta. generación, siendo estos los más evolucionados y creados con la finalidad de servir para la IA y el procesamiento de lenguajes naturales, LISP, PROLOG.

1.5.1 Lenguaje de programación más usados.

Hay múltiples lenguajes para programar, pero se destacan los más sustanciales basados en el índice TIOBE, siendo esta una investigación ejecutada por una corporación de Holanda que se dedica a analizar estos lenguajes y cuáles son los más utilizados a nivel global.

A continuación, los 6 segmentos más utilizados son:

- Java: Es el más actualizado y usado a nivel global, lo cual se debe a su simplicidad y legibilidad, está disponible en miles de millones de gadgets, lo que implica el 15% del absoluto del planeta en contraste con los diferentes dialectos. Se ha mantenido en las mejores situaciones durante todo el siglo XXI, lo que da una idea de cuán importante ha sido este lenguaje de programación en lo que usamos hoy. Se utiliza en varios fragmentos, como aplicaciones portátiles, aparatos de aprendizaje, páginas de contabilidad, entre otros, su simplicidad y puesto que es más reconocido a nivel mundial.
- Lenguaje de programación C: Surgió en 1970, al principio llamado "B". Con un alto nivel de utilización como java, 12%, es el segundo idioma más involucrado del planeta. Asimismo, su uso está muy variado y tiende a ejecutarse en la mayoría de los marcos de trabajo, en general se utiliza en las organizaciones.
- Python: Un lenguaje de programación de múltiples etapas y múltiples visiones del mundo, que además tiene una utilidad universal. Esto implica que defiende la programación basada en objetos, básica y utilitaria. Es un socio extraordinario para los novatos por su desenvoltura, significación y cercanía al idioma inglés.
- C++: Es una evolución del antes mencionado C. No obstante, esto está centrado para la evolución de aplicativos y software de mayor complejidad, lo cuales requieran, un tipo de implicaciones visuales. Por lo tanto, programas de diseño gráfico son la ejemplificación de lenguajes de programación.
- C#: Reconocido igualmente como lenguaje C Sharp, siendo la evolución de lenguaje C++ y C. Siendo un tipo de programación con un enfoque dirigido a objetos. Desarrollándose y creándose en el 2000, está destacado por la simplicidad, trabajando

con aplicativos bajo framework.net, teniendo una participación y aplicación a nivel global del 7%.

- Visual Basic. NET: Teniendo un galopante progreso en la cantidad de consumidores en la actualidad, está vinculado y reconocido como una herramienta con mayor manejabilidad y amigable, debido a que los conocimientos previos para su uso, no deben ser tan extensos como en el caso de C#, caracterizándose por la simplicidad, es muy manejado para herramientas con fines de sistematización de aplicaciones y procesos web.

Por lo tanto, los diversos tipos de lenguaje para la programación, fuera de dar una imagen de ser herramientas de complejidad en las que poco individuos pueden hacer uso o trabajar en torno a su manejo, son las mismas vitales para el adecuado ejercicio de lo que se conoce y maneja actualmente, rodeando internet y lo vinculado al mismo en la actualidad, los cuales, si son implementados en ámbitos educativos, resultan ser grandes aliados para los maestros y alumnos debido a su sencillez y buen manejo de la interfaz, así como por la familiaridad que poseen las nuevas generaciones con los aplicativos y las nuevas tecnologías.

Capítulo II

Programación orientada a objetos OOP

2.1 Programación orientada a objetos

Garrido (2015) afirma que:

Debido a ello, a la complejidad de los problemas, la programación evolucionó conceptualmente y surgió un nuevo paradigma llamado programación situada en objetos, que caracterizaba cada una de las sustancias de un programa en objetos que contenían información propiedades y prácticas estrategias. Estos artículos se agrupan en clases en función de características similares (p. 117).

Programación orientada a objetos, un paradigma de programación en el que los datos y las operaciones que se pueden realizar en esos datos se agrupan en unidades lógicas denominadas objetos.

Los objetos, por regla general, representan ideas en el espacio del programa, como alumnos, coches, teléfonos móviles, etc. La información que aborda las excentricidades de un artículo se denomina calidad, que es la parte estática del objeto, y las actividades que puede realizar un objeto se denominan técnicas, que son la pieza única del objeto. La programación orientada a objetos le permite simplificar la estructura y la lógica de programas grandes que involucran muchos objetos que interactúan.



Figura 13. Programación orientada a objetos. Fuente: Autoría propia.

2.2 Distinción de objetos y clases

2.2.1 Objetos.

El tipo de programación orientado a objetos (OPP) es una modalidad de programación en el ámbito de la informática, la cual ordena el esquema de un software relacionándolo a objetos o datos, en vez de lógica y funciones, de manera que un ente logra definirse un conjunto de datos amplios, los cuales mantienen comportamientos y atributos determinantes y únicos (Inca, 2012).

2.2.2 Clases.

Partiendo de las diversas clases, pueden desarrollarse distintos objetos, donde cada uno puede poseer un valor propio determinado según los atributos del mismo (Joyanes, 2008).



Figura 14. Atributos del objeto. Fuente: Recuperado de <https://aprendeconalf.es/docencia/python/manual/objetos/>

2.2.3 Mensajes.

La comunicación e interacción de los objetos es ejecutada a través de mensajes, donde si se desea que un objeto ejecute una actividad o mensaje se puede adquirir información establecida como parámetros de manera adicional, ya que cuando un objeto percibe una información o mensaje ejecutará de acuerdo al mismo una operación o método determinado, teniendo como características para esto que, para el tránsito de dicha información es necesaria el componente del mensaje, el objeto del mismo, la metodología, la cual debe ser ejecutable como una respuesta y los parámetros requeridos para la ejecución del método.

2.3 Principios

En la fundamentación principal para la comprensión en cómo se modela la realidad de un programa, bajo el paradigma de la orientación enfocada a objetos, los principios básicos son, abstracción, encapsulamiento, modularidad, jerarquía, polimorfismo y paso de la información o mensajes.

2.3.1 La herencia.

Tratar de no copiar código en clases con prácticas comparativas, caracterizando estrategias normales en una clase madre y técnicas específicas en clases para hijas.

La herencia permite caracterizar nuevas clases dependientes de las existentes para reutilizar el código, creando posteriormente un orden de clases dentro de una aplicación. En la remota posibilidad de que una clase obtenga de otra, adquiere sus créditos y técnicas y puede agregar nuevas propiedades, estrategias o repensar las adquiridas. Herencia, está determinada su función por un parámetro base, teniendo propiedades únicas caracterizadas por un antecesor, es decir, es una forma de simplificación de diseños, evitando la duplicación de códigos no debiendo codificarse metodologías ya planteadas, determinándose así el acto de toma de propiedades características de un común (Joyanes, 2008).

Estas clases no se mantienen aisladas, relacionándose entre ellas, estableciéndose una estructura jerárquica de clasificaciones, por lo que los objetos heredan atributos y comportamientos de las clases pertenecientes a cada uno, teniendo que la herencia ordena y promueve la encapsulación y polimorfismo al permitir que dichos objetos se definan y creen como tipologías especiales de estructuras ya existentes.

Pudiendo vincular y ampliar el funcionamiento, no debiendo implementarse nuevamente, donde comúnmente se desarrolla mediante la agrupación de objetos de

acuerdo a su clase, agrupándose en árboles o celosías, las cuales mantienen un comportamiento, teniendo que al heredar un objeto diversas clases es referido como herencia múltiple.

2.3.1.1 Tipos de herencia en Java.

Java, como la mayoría de los dialectos de programación actuales, tiene varios tipos de legado que podemos usar para hacer que nuestro programa sea significativamente más productivo agregando cualidades o propiedades de varias clases.

Lo que debemos recordar es que debe haber una superclase, como encontraremos en los tipos de legado adjuntos:

- Herencia extraordinaria: donde las subclases adquieren las cualidades de una sola superclase.
- La herencia escalonada: una clase determinada adquirirá una clase base y, además, la clase inferida también actuará como la clase base de otra clase.
- Legado progresivo: una clase se completa como una superclase (clase base) para más de una subclase.
- Varios heredados (a través de interfaces): una clase puede tener más de una superclase y adquirir atributos de la relativa multitud de clases principales. Sin embargo, Java no mantiene varios legados con clases, por lo que para lograr esto necesitamos utilizar interfaces.
- Mitad y mitad heredados (a través de interfaces): es una combinación de al menos dos tipos anteriores de herencias. Dado que Java no mantiene un legado diferente con las clases, la mitad y la mitad del legado tampoco son prácticas con las clases; sin embargo, como en el modelo anterior, podemos lograr resultados similares a través de interfaces.

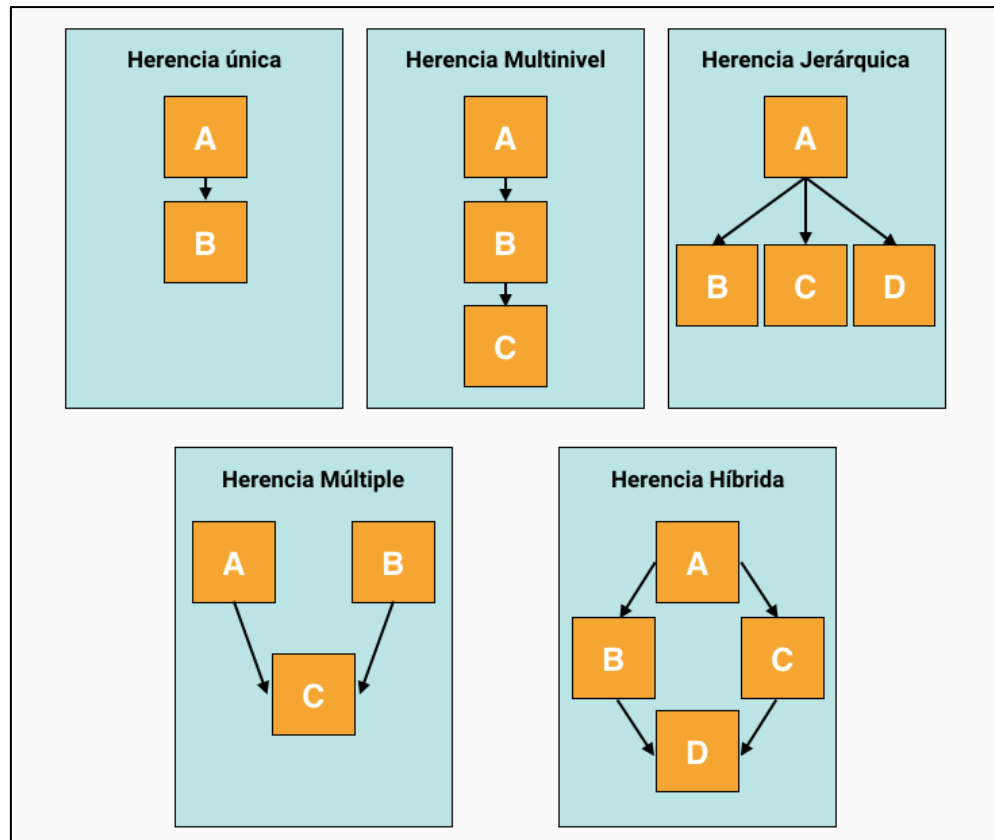


Figura 15. Tipos de herencia en Java. Fuente: Recuperado de <https://ifgeekthen.nttdata.com/es/herencia-en-programación-orientada-objetos>

En conclusión, la herencia abre un mundo de situaciones y composición que nos ayuda a manejar el desarrollo complejo de manera más eficiente, mostrando un código ordenado y estructurado que no solo es fácil de leer, sino que también se puede mantener en futuros ciclos de desarrollo.

2.3.2 Polimorfismo.

En la programación organizada por objetos, el polimorfismo es la capacidad de un objeto de una clase para dar reacciones únicas y libres que dependen de los límites (varias ejecuciones) utilizados durante el conjuro. Es decir, un artículo, como elemento, puede contener varios tipos de cualidades durante la ejecución del programa. En cuanto al polimorfismo, está referido a las distintas formas que puede adoptar un objeto, donde dicha propiedad faculta a que el mismo pueda presentar distintos comportamientos de acuerdo a

la función, determinándose por el contexto donde se encuentre. Por ejemplo, dependiendo de los argumentos recibidos, un método puede tener diferentes implementaciones, donde también la percepción de distintos parámetros puede darse, para la ejecución de una operación, desarrollándose distintas acciones de acuerdo a la abstracción a la cual se vincule. El método del polimorfismo OOP implica la capacidad de adoptar más de una forma. Una actividad puede mostrar varias prácticas en distintas ocasiones. La conducta se basa en los tipos de información utilizados en la actividad. El polimorfismo se utiliza ampliamente en el uso de legado.



Figura 16. Ejemplo de polimorfismo. Fuente: Recuperado de <http://www.itnuevolaredo.edu.mx/takeyas/apuntes/poo/Apuntes/04.-Polimorfismo.pdf>

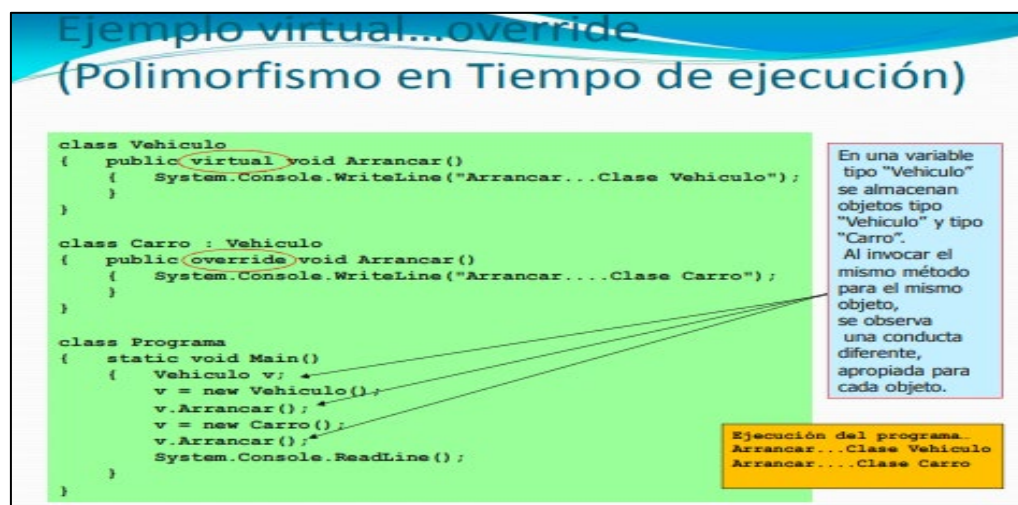


Figura 17. Ejemplo de polimorfismo en ejecución. Fuente: Recuperado de <http://www.itnuevolaredo.edu.mx/takeyas/apuntes/poo/Apuntes/04.-Polimorfismo.pdf>

2.3.3 Abstracción.

La abstracción se centra en la visión externa del objeto y separa el comportamiento concreto del objeto, esta división que realiza se denomina barrera de abstracción y se logra aplicando el principio de mínimo compromiso.

Por lo tanto, la abstracción se vincula a la mente humana, ya que se modeliza como objeto la realidad, teniendo la búsqueda de esquemas, paradigmas o ejemplos, donde exista similitud de la realidad y la implantación de la misma a través de objetos en el programa buscando la simulación de la función de los mismos en el aspecto tangible o real.

Existe un gran alcance de deliberaciones que existen, desde objetos que modelan contenidos excepcionalmente cercanos hasta objetos que no tienen una buena excusa para existir, lo notaremos rápidamente.

- Abstracción de entidades: Es un artículo que aborda un modelo valioso de un elemento ideal.
- Abstracción de acciones: un elemento que aborda un montón de tareas y cada una de ellas desempeña roles de un tipo similar.
- Abstracción de máquinas virtuales: objeto que agrupa tareas, cada una de ellas virtuales, utilizadas por un nivel más elevado de control o actividades, entre ellas podríamos hablar de un circuito.
- Abstracción de coincidencia: un elemento que almacena un montón de tareas que son aleatorias entre sí.

La abstracción contemplada desde la perspectiva de la programación situada en el objeto comunica las cualidades fundamentales de un artículo, que reconocen el artículo de los demás. Además de reconocer objetos, proporciona puntos de corte teóricos. Así que se podría decir que la ejemplificación aísla lo fundamental de lo trivial dentro de un elemento. Suponiendo que un artículo tiene una mayor cantidad de cualidades de las

necesarias, será difícil de utilizar, cambiar, fabricar y ver, especialmente cuando se trata de una estrategia numérica con un final fragmentario.

Crea una ilusión de sencillez, ya que limita la cantidad de cualidades que caracterizan a un artículo.

Modelo 3: La administración de un taller mecánico necesita un marco para controlar los vehículos que ingresan a sus oficinas. Para esta situación, las cualidades fundamentales de la clase de vehículo son: marca, modelo, color, falla identificada, nombre del propietario, dirección del propietario, número de teléfono del propietario.

Conceptos OOP: Abstracción

- La abstracción **no es única**:

Un coche puede ser...

- Una cosa con ruedas, motor, volante y pedales (conductor)
- Algo capaz de transportar personas (taxista)
- Una caja que se mueve (simulador de tráfico)
- Conjunto de piezas (fabricante)



Figura 18. Ejemplo de abstracción. Fuente: Recuperado de <https://en.ppt-online.org/323450>

2.3.4 Encapsulamiento.

Decimos que el encapsulamiento en programas de computadora de escritura situados en objetos es el punto en el que limitamos el acceso o damos admisión confinada de una propiedad a los componentes que una parte necesita y no a más.

El componente más ampliamente reconocido del epítome son las clases, donde tipificamos y englobamos las dos estrategias y propiedades.

Otra ilustración extremadamente normal de epítome son los captadores y definidores de las propiedades dentro de una clase. Por supuesto que nos dan la estima ordinaria, pero podemos alterarlos para que cambie.

Cada objeto se desacopla desde una perspectiva externa, es un módulo característico y toda la aplicación se reduce a un total o rompecabezas de artículos. La segregación protege la información relacionada con un artículo contra su alteración por parte de quien no se reserva el privilegio de acceder a él, borrando impactos auxiliares y colaboraciones.

De esta manera, el cliente de la clase puede eludir la ejecución de las técnicas y propiedades para enfocarse solo en la mejor manera de utilizarlas.

Por otra parte, se evita que el cliente cambie su estado de manera inesperada y descontrolada.

Modularidad, está comprendida como la propuesta de la aplicación en diversos módulos distintos donde cada uno, conforme la estructura, mantenga un sentido único, teniendo que la fragmentación reduce el nivel de dificultad para la resolución del problema a nivel general, el cual es la finalidad del programa o respuesta, debido a que la solución del problema como conjunto deriva en una dificultad de menor moderación, lo cual facilita la comprensión del mismo. El encapsulamiento caracteriza los niveles de entrada para los componentes de esa clase. Estos niveles de entrada caracterizan las libertades de entrada

de la información, permitiéndonos llegar a la información a través de una estrategia para esa clase específica, de una clase adquirida, o incluso de alguna otra clase.

Hay tres grados de acceso:

- público: elementos, en igualdad de condiciones, que pueden llegar a la información o estrategias de una clase que se caracteriza por el nivel libre. Este es el grado más mínimo de seguridad de la información.
- asegurado: el acceso a la información se limita a elementos de clases adquiridas, es decir, capacidades que son individuos de esa clase y todas las subclases.
- privado: el acceso a la información se limita a las técnicas para esa clase específica.

Este es el nivel más elevado de seguridad de la información.

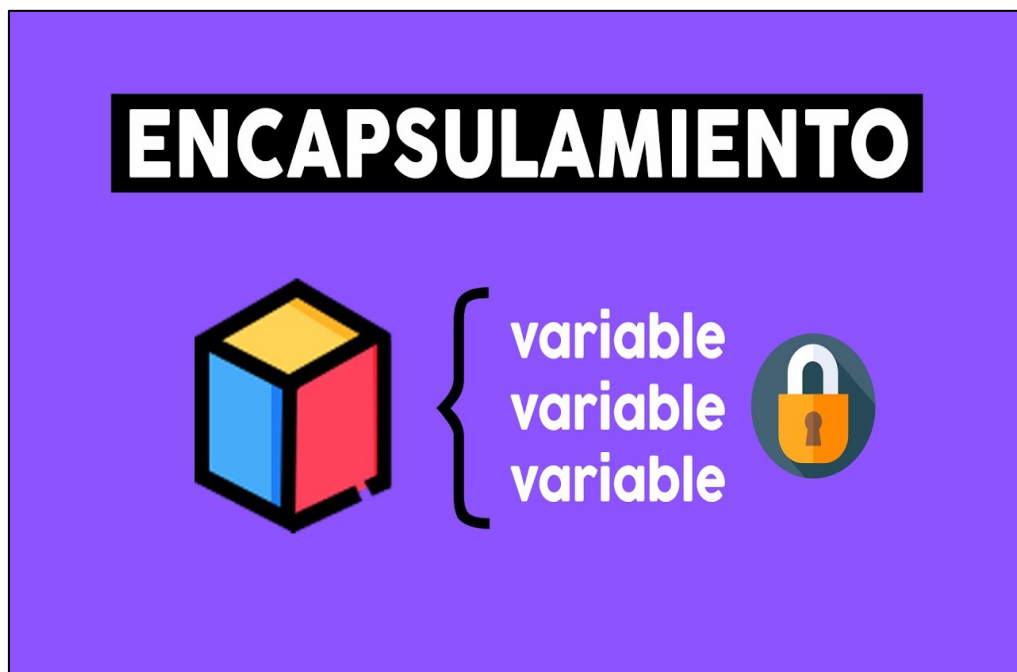


Figura 19. Encapsulamiento. Fuente: Recuperado de <https://www.youtube.com/watch?v=MHQZyCniPkk>

Ventajas y desventajas:

Las ventajas de la programación orientada a objetos es que:

- La misma promueve el uso reincidente y desarrollo de los códigos.

- La creación y desarrollo de sistemas de mayor complejidad se permite.
- El desarrollo de la programación mantiene una similitud con la realidad, permitiendo la interacción entre participantes.
- El desarrollo del software necesario para su implementación se agiliza de acuerdo a la programación.
- El trabajo en equipo es facilitado de acuerdo a la programación utilizada, teniendo en cuenta que la reconstrucción de la realidad puede determinarse a través de Java.
- La misma puede proporcionar herramientas y fundamentaciones en donde los programadores pueden representar la realidad a través del ámbito virtual, manteniendo similitudes e interacciones dinámicas.

Capítulo III

Utilidades para el desarrollo y prueba de programas

3.1 Compiladores

El compilador es el programa en el ámbito informático encargado de la traducción de un programa escrito, a través del lenguaje de programación o código fuente, a otro tipo de lenguaje de alto nivel (COBOL, PASCAL, BASIC, C, etc.), medio o bajo (intermedio o código máquina) como C / C. Por otro lado, un programa es el conglomerado de orientaciones codificadas detalladas que se transmiten a una computadora para realizar o realizar ciertas actividades.

La relevancia de los compiladores reside en las aplicaciones informáticas, siendo la base para la programación en cualquiera de las plataformas. Además, a partir del mismo código fuente, un compilador tiene la capacidad de producir código de máquina en múltiples plataformas al mismo tiempo (Merino, 2015).

Por lo tanto, es aceptado que para el desarrollo de un programa es necesario un compilador, de manera que el mismo, en su estructura y desarrollo, es un programa de computadora.

Es importante no confundir el compilador con un intérprete. Al igual que el compilador, el intérprete convierte un programa siendo fuente que está escrito al lenguaje

de alto nivel, pero con la diferencia de que cada instrucción se ejecuta inmediatamente sin generar un programa en lenguaje necesario para máquina (Schmidt y Parra, 2003).



Figura 20. Proceso de compilador. Fuente: Recuperado de <https://javacurso.wordpress.com/compilar-java/>

3.1.1 Primer compilador.

La doctora en Matemáticas y General, Grace Hopper, del Ejército de los EE. UU., creía firmemente que, si el proceso de programación se vuelve más amigable y comprensible para los no matemáticos, las computadoras pueden llegar a una audiencia más amplia y comercial.

Donde su percepción se basaba en que en los ordenadores era necesaria la escritura en inglés, por lo que alrededor del año 49, de acuerdo a su convencimiento, desarrolló un método, el cual podía traducir simbología matemática en códigos de instrucciones binarios, siendo compresibles por máquinas, teniendo que la base de leguajes de programación actualmente se ha establecido a partir de ese momento.

En el caso del primer programa informático de la historia, se acepta que fue desarrollado por Ada Lovelace, quien fue escritora y matemática, ya que sus investigaciones se basaron en el desarrollo de un algoritmo, el cual tenía como finalidad ser procesado de manera directa por una máquina.

3.1.2 Intérpretes.

Es un programa encargado del procesamiento del código fuente, establecido para un proyecto de un software, el cual es ejecutado durante su desarrollo, actuando como interfaz entre procesador y dicho proyecto. Por lo tanto, un intérprete procesará el código de manera sistemática, teniendo en cuenta que capta, analiza y formula las secuencias lineales para el procesador, siendo esto aplicado para las secuencias repetidas, las cuales se volverán a ejecutar a medida que reaparezcan en el código establecido, mientras que en el procesamiento del código fuente del software, el intérprete, utiliza su propia biblioteca interna: siempre que una línea de código se traduzca al comando correspondiente legible, enviándose de forma directa al procesador.

Por lo tanto, la conversión en su proceso no terminará hasta el momento que se hayan explicado todos los códigos. Si ocurre un error durante el procesamiento, solo terminará prematuramente, lo que simplifica enormemente la resolución de problemas porque la línea correspondiente al código en cuestión es capturada al momento siguiente de que ocurre el error.

3.2 Depuradores

Son programas que permiten la detección y diagnóstico de fallas en programas de índole informático. Teniendo como finalidad este tipo de herramientas el asegurar, en un periodo prolongado de tiempo, que el software de acuerdo a su funcionamiento sea ejecutado en las plataformas y dispositivos para los que se encuentra diseñado. Debido a esto, una gran cantidad de depuradores analizan el código fuente y también las formas en que interactúa con el sistema operativo a través del cual es ejecutado con elementos del hardware correspondiente; por lo que el depurado ocurre durante la ejecución del

programa, de manera que no existe la necesidad de interrumpir la ejecución para que el análisis sea ejecutado (Jiménez, Martín y Sierro, 2010).

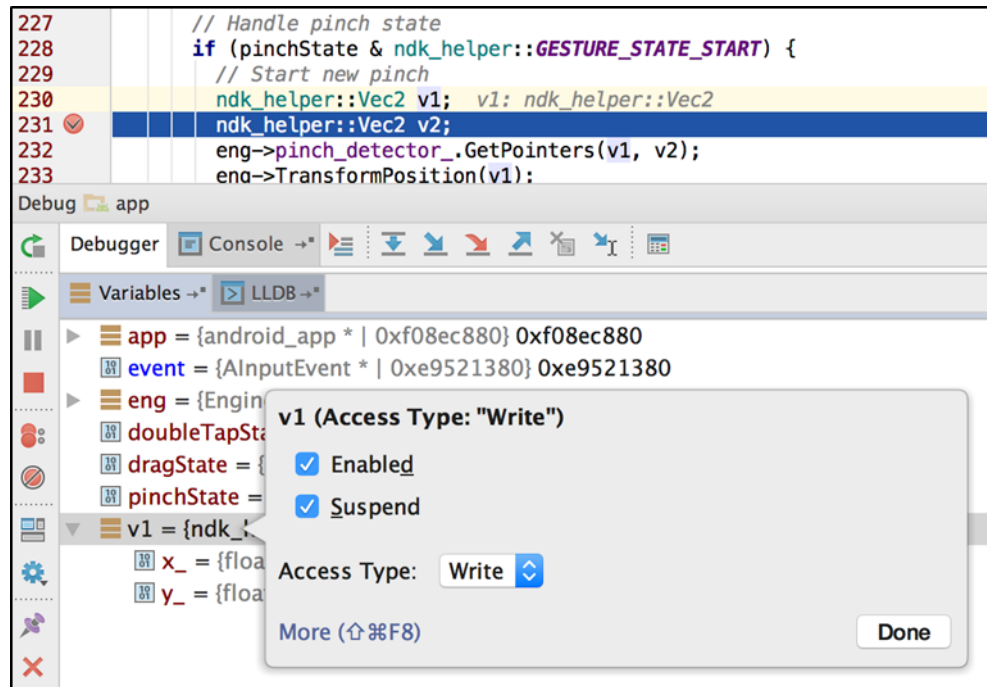


Figura 21. Proceso de depuración. Fuente: Recuperado de <https://developer.android.com/studio/debug?hl=es-419>

Por su parte, la composición de la palabra *debuggin* (depuración) se compone por un prefijo y la denotación bug, siendo entendida como error, teniendo que el término se ha utilizado en el desarrollo de software durante décadas para denotar fallas o errores de cualquier tipología en programas. En la práctica, dicha denotación se ha utilizado algunas veces durante el siglo XIX para la descripción de fallas. Debido a esto, la eliminación de fallas de software también se conoce como corrección de errores.

En cuanto a sus funciones, cualquier software no se encuentra a cabalidad libre de fallas o errores, la optimización diaria es una tarea fundamental. Por lo que la problemática no se enfoca en la identificación de procesos que sean defectuosos durante la ejecución, sino que se vincula directamente a una causa en concreto. Por ejemplo, un soporte ubicado incorrectamente o un operador utilizado incorrectamente es suficiente para que el

programa deje de funcionar correctamente. No obstante, cuando el mismo tiene que enfrentarse con un número elevado de líneas, la búsqueda de forma manual entre caracteres no es rentable. Los depuradores modernos, por otro lado, además de la detección de problemas, proporcionan datos específicos sobre el tipo de fallas o errores y frecuentemente indican la zona o línea de código donde proviene la falla. Los depuradores normalmente revisan la codificación del software paso a paso, debiendo el software establecer los denominados puntos de interrupción. Por lo que el depurador examinará dichos puntos y utilizará a los mismos para evitar que el software se ejecute en esos puntos (González, 2013).

Por lo tanto, se puede analizar los estados de programa en periodos de tiempo determinados, comprobándose dicho estado fácilmente, de manera que si las variables implícitas en el proceso arrojan resultados esperados, se puede hacer la ejecución del programa partiendo del punto de interrupción utilizado para el descarte de posibles fallas o causales de error.

Así, dichos puntos no deben ser correctos; sin embargo, a algunos depuradores les es permitida la asignación de puntos de interrupción de acuerdo a las condiciones específicas necesarias por cada uno, facultándose el programa para detenerse de acuerdo a señales específicas. Por ejemplo, si una cantidad de procesos se ejecuta con éxito 15 veces, pero demuestra un error de programación en la ejecución número 16, el depurador puede tomar acción directa sobre el problema si se ha establecido la condición apropiada. En consecuencia, la cantidad de ciclos que se han ejecutado, no obteniéndose resultados, no deben ejecutarse nuevamente.

Dentro de la función de los depuradores, también se encuentra el estudio y análisis de datos, teniendo en cuenta el contenido de variables que intervienen, memorias o registros (González, 2013).

Por otra parte, los *debuggers* tienen la capacidad de modificar memorias como la principal o los registros encontrados en el procesador.

Teniendo que son una forma para que los programadores vean rápidamente las fallas que se desarrollan durante la ejecución del programa. Por lo tanto, los encargados de la programación a menudo usan varios comandos de impresión que requieren que el programa se recompile, de acuerdo a la colocación de comandos nuevos al código. Luego de la depuración, todos los comandos de impresión deben eliminarse, donde durante el proceso algún comando puede de ser el caso olvidado o perdido al eliminarlo, lo que provocaría errores a posterior.

El conocimiento de las técnicas de depuración necesarias ayuda a los programadores al encuentro eficaz y rápido de errores en la codificación establecida, determinando así puntos de interrupción y deteniendo la ejecución en momentos determinados, no teniendo la necesidad de imprimir comandos en mayor cantidad, determinándose que no es necesario volver a compilar el programa cada vez que se utiliza un depurador, funcionando de igual forma con programas de código cerrado, los cuales no permiten acceso al código fuente.

Por otra parte, otro conjunto de beneficiarios está referido a los administradores de sistema, los cuales comúnmente ejecutan y despliegan distintos programas desarrollados sobre sistemas de producción, encontrándose que el comportamiento del programa no es el esperado, afectando en nuestro caso al área académica debido a que durante las pruebas la ejecución del mismo no arroja errores al sistema pero presenta fallas relacionadas a los sistemas de producción, por lo que dichos administradores mantienen la responsabilidad de identificación del problema, donde el encuentro e informe a los programadores depende directamente de los mismos, por lo que los depuradores resultan de utilidad en el descubrimiento de los lugares y razones de la falla del programa.

La siguiente reunión de expertos en TI para sacar provecho de los depuradores son los analistas de seguridad, que necesitan estos aparatos constantemente. Hay varios tipos de áreas en las que un científico de seguridad utiliza depuradores de manera constante, como averiguar o realizar tareas de investigación de malware. Para sus propósitos, tener la información vital y las habilidades con diferentes depuradores es una necesidad absoluta, con el argumento de que, si no, se quedarían fuera de una tarea.

De igual manera, los investigadores de seguridad se benefician directamente de los depuradores debido a la necesidad de las herramientas, ya que existen diversos tipos de dominios; donde los mismos utilizan depuradores en torno a una base diaria, siendo necesario para el análisis y detección de malware como para la ingeniería reversiva, siendo necesario el conocimiento y habilidad en torno a esto, para una ejecución adecuada del trabajo. Por lo que distintos grupos de profesionales se benefician del conocimiento en depuración, teniendo que la habilidad del individuo es invaluable en la ejecución eficiente de tareas, por lo que los poseedores de técnicas de depuración idóneas son más competentes en el mercado laboral y solicitados por una cantidad significativa de organizaciones. Dentro del contexto académico, utilizar estos depuradores resultan ser buenos aliados tanto para el alumnado como para el profesorado, los cuales nos ayudarán a evitar posibles errores al momento de un dictado de clases, especialmente en la nueva normalidad originada por la pandemia.

3.3 Prueba de programas en lenguaje Java

3.3.1 Concepto.

La Prueba de Software es un arte entre las actividades de verificación y validación, esto porque se concentra entre un 50 a un 75 por ciento del tiempo de las fases de desarrollo y mantención del ciclo vital del mismo.

La prueba es denominada al proceso para ver si hay una diferencia entre el comportamiento esperado (que cumple con los requisitos) y el comportamiento que realmente se observa. Para ello se utilizan casos de prueba, que son el conjunto de entradas y resultados esperados para un componente con el objetivo de provocar fallas y detectar defectos.

Las pruebas unitarias son importantes para reducir las pruebas posteriores, pero, debido al tiempo y el costo que representan, rara vez se practican porque los casos de prueba generalmente se generan manualmente. En el caso de un sistema construido bajo el modelo enfocado a objetos, las pruebas unitarias se enfocan en clases, y una de las técnicas utilizadas son las técnicas de McGregor y Sykes. Con base en esta tecnología y el uso de JML (Java Modeling Language) para el diseño de contratos, se diseñó y construyó un prototipo funcional de la herramienta para ayudar a generar casos de prueba unitarios.

3.4 Tipo de pruebas

3.4.1 Pruebas unitarias.

Con estas, los errores se pueden detectar antes. Si no hay pruebas unitarias, los errores no son detectados hasta etapas avanzadas (como, pruebas de sistema, integración o incluso versiones beta). La prueba unitaria regular puede, en última instancia, ahorrar tiempo y dinero, lo que significa realizar pruebas al aislar la clase de las interacciones de otra clase (Viera, 2019).

Se establecen como principios FIRST necesarios para la escritura y desarrollo de pruebas unitarias de la siguiente manera:

- F: Referido a la rapidez que deben ser ejecutados los test.
- I: Denotándose como los test realizados sobre una clase sin mantener interacciones con otra.

- R: Referente a la orden de acuerdo a la ejecución, teniendo que los mismos no deben influir en el final.
- S: La ejecución de los test debe realizarse de manera automática.
- T: Desarrollándose al tiempo que se crea dicho software.

Por su parte, en el caso de Java, la elección puede darse entre distintas opciones, donde cada una de las mismas es JUnit, siendo esta la de mayor consolidación.

3.4.2 Pruebas de integración.

Esta referido a la comprobación de las distintas interfaces donde se vinculan los grupos de subsistema o componentes, para asegurarse de que los datos o mensajes sean transmitidos de manera adecuada de acuerdo a los requisitos preestablecidos (Viera, 2019).

En las pruebas unitarias se requiere la creación de módulos auxiliares para la simulación de acciones vinculadas a los componentes, determinados por el probador debido a que deben ser conductores para el establecimiento de prerequisites necesarios, llamar a los componentes probados y verificar los resultados de la prueba, prueba unitaria y los tipos de pruebas de integración se utilizan a menudo en combinación.

Los tipos fundamentales de integración son los siguientes:

Integración incremental

Es la combinación del componente con el conjunto de los mismos ya experimentados y la cantidad de componentes a prueba se incrementa progresivamente.

Las pruebas incrementales se vinculan a problemas que surgen en la incorporación de características o componentes validados de manera previa donde se deben muy probablemente a la relación entre las interfaces. Se distinguen las siguientes estrategias de integración. De arriba hacia abajo, el primer integrante o componente que será desarrollado y probará es el primer factor jerárquico (A); mientras que los integradores de

nivel inferior reemplazan por componente de soporte para la simulación de componente retirado. Por lo tanto, no es requerido componente conductor. Siendo una ventaja en la aplicación de la estrategia debido a que la interacción de las interfaces de diverso componente es probada de manera frecuente y temprana.

De abajo hacia arriba, en este caso, el nivel más bajo de componentes (E, F) son creados primero, mientras que los conductores se crean para la simulación del componente que los llama. Seguidamente, se da el desarrollo y prueba de los de nivel superior siendo B, C y D. Finalmente, estos componentes se combinan con lo que los llama (A). Rara vez se necesitan componentes auxiliares.

Estrategias combinadas: Dentro de la utilidad de la aplicación de forma conjunta de las estrategias descritas, los elementos del sistema se desarrollan bajo enfoque de arriba hacia abajo, mientras que los componentes de mayor dificultad o críticos son desarrollados de forma contraria, siendo necesaria una planificación coordinada y rigurosa, para que de forma individual los componentes se encuentren en una etapa media.

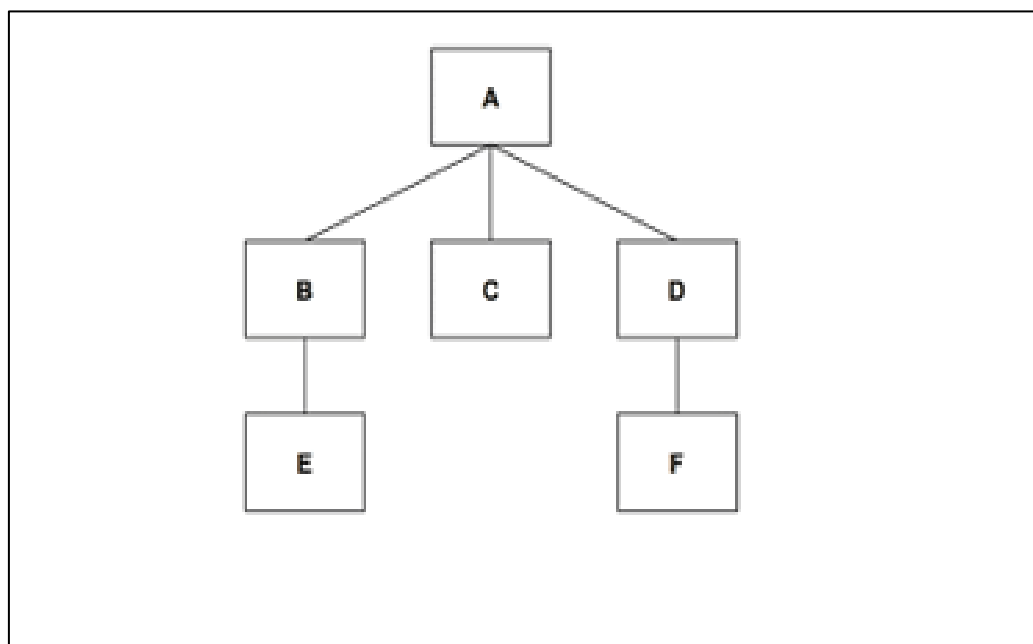


Figura 22. Integración incremental. Fuente: Autoría propia.

Integración no incremental; Es ejecutada de manera que los componentes son probados por separado, integrándose al final de los procesos todo el conjunto, que resulta así mediante la ejecución de pruebas adecuadas, siendo denominada esta forma de integración Big Bang.

3.4.3 Pruebas de sistemas.

Están referidas con la finalidad de ejercitar de manera adecuada al sistema, comprobando el tipo de integración del sistema de forma global, donde se verifica el adecuado funcionamiento de la interfaz entre subsistemas que lo conforman, así como de los sistemas vinculados a la comunicación de los mismos (Aristegui, 2010).

Por lo tanto, son test de información completos que permiten comprobar el sistema de manera conjunta con otro sistema, donde se relaciona para la verificación de funcionalidades técnicas de acuerdo a su cumplimiento, dando una visión del comportamiento de acuerdo a la producción en el entorno donde se ejecuten.

Cuando los componentes individuales han sido analizados e integrados, el sistema se prueba como un todo.

En esta fase se pueden distinguir diversos tipos de pruebas de acuerdo a la finalidad, donde:

Las pruebas funcionales. Se dirigen al aseguramiento de la información, donde el sistema ejecute de manera adecuada las funcionalidades, las cuales se detallan conforme a las especificaciones.

Prueba de comunicación. Está determinado por la interfaz vinculada entre componentes, determinando su funcionamiento de manera local y remota.

Pruebas de rendimiento. Se basan en la determinación de tiempos de respuestas, vinculados a los tiempos determinados en las especificaciones de sistema.

Pruebas de volumen. Se vincula al análisis del funcionamiento de acuerdo al volumen de datos esperado durante cargas de trabajo elevadas.

Prueba de sobrecarga. Se basa en la verificación del funcionamiento de los sistemas al límite de recursos disponibles, sometiénolo a cargas de datos de manera masiva, teniendo como finalidad los puntos donde el colapso del sistema o el comienzo de operaciones se desarrolla a un nivel menor de los parámetros preestablecidos.

Pruebas de disponibilidad de datos. Se basa en la demostración de recuperación del sistema, frente a cualquier eventualidad negativa, tanto de manera física como lógica sin existir el compromiso de datos.

Pruebas de facilidad de uso. Las mismas se ejecutan en torno a la adaptabilidad del sistema en relación con la necesidad del usuario, asegurando que el mismo se organiza de acuerdo a los volúmenes de trabajo habituales del usuario, así como la determinación de facilidades que aporta al momento de la introducción de datos y obtención de resultados esperados.

Pruebas de operación. Se vinculan a la comprobación de implementaciones idóneas en el procedimiento de las operaciones, donde el control de trabajo, planificación y arranque determinan la operación adecuada.

Pruebas de entorno. Se vincula a la relación interactiva del sistema con otros en un entorno similar.

Pruebas de seguridad. Es la verificación de mecanismos de acceso y formas de control al sistema, donde se previene la alteración de datos.

3.4.4 Pruebas de validación.

La finalidad de estas es la obtención de información de utilidad para la validez y ejecución de algoritmos establecidos, donde se asume que el software para esta etapa ya se

ha verificado, teniendo en cuenta que se encuentra libre de fallas o errores relacionados a los tiempos de ejecución, mas no de errores lógicos.

La metodología propuesta para ellos es la que acompaña:

- Piense en organizaciones algo pequeñas (confiables y problemáticas).
- Decide físicamente el registro de calidad inquebrantable (utilizando, por ejemplo, el cálculo de la especificación de estado completo o cuando sean posible, científicamente).
- Contraste los resultados dados por los cálculos.

3.5 Ventajas

Entre las ventajas, destaca:

- La calidad de código mejora, puesto que podemos detectar errores en una etapa más temprana de desarrollo y de forma más rápida.
- Puedes trabajar de una forma más ágil, ya que favorece la integración y facilita cambios.
- Los test pueden funcionar como documentación y ejemplos de nuestras clases.
- Reduce el costo de mantenimiento del proyecto.
- A través del desarrollo guiado por pruebas, nos ayuda a mejorar el diseño de nuestro software.
- Estos son algunos beneficios que trae el desarrollar pruebas a nuestros proyectos. Como podrás ver, no importa si el tiempo de desarrollo aumenta un poco, pues a la larga son muchos los beneficios que trae consigo.

Aplicación didáctica

Sesión de aprendizaje

“Representamos con números enteros los cambios de temperatura y comprobamos nuestras respuestas haciendo uso de un programa creado en Java”

I. Datos informativos

Área : Matemática
 Unidad : Uso cotidiano de los números enteros en mi comunidad.
 N° de sesión : 1
 Fecha : 24 - 05 - 2021
 Duración : 90 minutos
 Docente : Lilian Georgette Aviles Gutierrez

II. Propósitos de aprendizaje

Competencia: Resuelve problemas de cantidad.
 Capacidades: Traduce cantidades a expresiones numéricas.
 Usa estrategias y procedimientos de estimación y cálculo.
 Campo temático: Operaciones con números enteros.

III. Enfoques transversales

Enfoque priorizado: Búsqueda de una formación integral
 Acciones Observables: Disposición a adquirir cualidades para un mejor desempeño.

IV. Propósito de la sesión	
Determinar las variaciones en la temperatura ambiental de una comunidad haciendo uso de los números enteros (Z) y sus operaciones.	
V. Recursos y materiales	
Cuaderno de trabajo de Matemática 1 “Resolvamos problemas”. Fundamentos de la programación (2008) Madrid; Editorial Interamericana de España. Paleógrafos, plumones, cartillas, láminas, tabletas.	
VI. Secuencia didáctica	
Inicio	Tiempo: 20
La docente saluda y da la bienvenida a los estudiantes, establece las normas de convivencia para un trabajo ordenado y responsable dentro del aula. Luego presenta la siguiente situación:	
1. En un grupo de personas andinas, un día se registra la temperatura natural de 5 grados centígrados sobre cero. Después de dos días, se registró en 0 grados centígrados. En cualquier caso, se proyecta que a los cuatro días la temperatura bajará a 5 grados centígrados bajo nada, provocando condiciones que influyan en el bienestar de las personas y puedan introducir alguna infección respiratoria. ¿Cuáles son las variedades de temperatura registradas? En caso de que siga disminuyendo de esta manera, ¿en qué cantidad podría cambiar la temperatura ocho días después de que se registró la estimación primaria?	
La docente plantea algunas interrogantes a los estudiantes para recoger los saberes previos:	
▪ ¿Qué significa encontrarnos a temperaturas por encima del cero?	
▪ ¿Qué significa encontrarnos a temperaturas por debajo del cero?	

A continuación, la docente presenta la ficha de trabajo N° 1, para así realizar el conflicto cognitivo.

Se pide a los estudiantes que trabajen de manera ordenada, para que después, mediante lluvia de ideas, la docente pida la participación de los estudiantes con la pregunta de dicha ficha.

Luego, la docente acoge las respuestas dadas por los estudiantes sin juzgar la validez o no de las mismas, y a partir de ahí señala el propósito de la sesión:

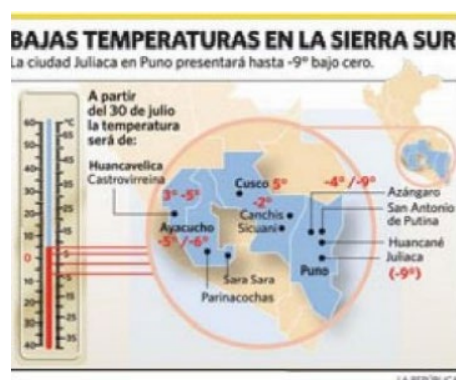
- Determinar las variaciones en la temperatura ambiental de una comunidad haciendo uso de los números enteros (Z) y sus operaciones.

Desarrollo

Tiempo: 50

El educador presenta las diapositivas adjuntas y aclara que hay muchas situaciones en las que rastrearemos números lamentables.

Temperatura bajo nada: En varios lugares del Perú se alcanzan temperaturas de 10°C ; por ejemplo: Apurímac, Ayacucho y Moquegua.



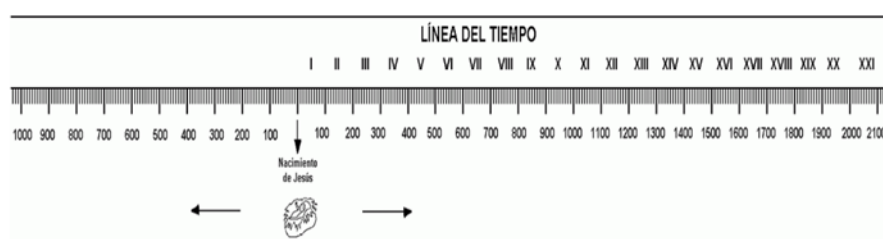
Fuente: Recuperado de <https://diariouno.pe/temperatura-desciende-a-20-grados-en-la-sierra-sur/>



Fuente: Recuperado de <https://elcomercio.pe/noticias/heladas/>

Elevaciones: Utilizamos las palabras "por encima del nivel del océano" y "por debajo del nivel del océano" cuando hablamos de la altura de ciertas áreas del Perú, pendientes, lagos o lugares. En el punto en el que decimos "nivel a la deriva" significamos "cero típico".

Fechas: para mostrar los números completos en la recta numérica, utilizamos las fechas a. C y d. C.



Posterior a ello, la docente pide a los alumnos que abran sus cuadernos de trabajo (“Resolvamos Problemas”) para que, con ayuda de la docente, procedan a desarrollar los ejercicios planteados en dicho material.

A manera de práctica, la docente pide a los alumnos que resuelvan 5 ejercicios del libro “Resolvamos Problemas” indicados por la docente. Los estudiantes resolverán dichos ejercicios, que será evaluado de manera individual. La docente les indica que tendrán un tiempo máximo de 30 minutos.

Concluido el tiempo, los estudiantes envían a la docente una fotografía de su libro de trabajo donde se encuentra la solución de los 5 ejercicios dejados por ella.

Cierre	Tiempo: 20
Metacognición:	
Se realizan preguntas sobre lo que se aprendió en la clase y en qué situaciones se aplicarían.	

Evaluación

Los estudiantes realizarán un vídeo donde presenten la resolución y explicación de una situación de contexto real sobre números enteros.

Referencias

Chavarri, E. (2004). *El ciclo hidrológico y la clasificación climática del Perú*. Recuperado de http://tarwi.lamolina.edu.pe/~echavarri/clase_ii_ciclo_hidrologico_clasif_clima_peru.pdf.

Docente de área

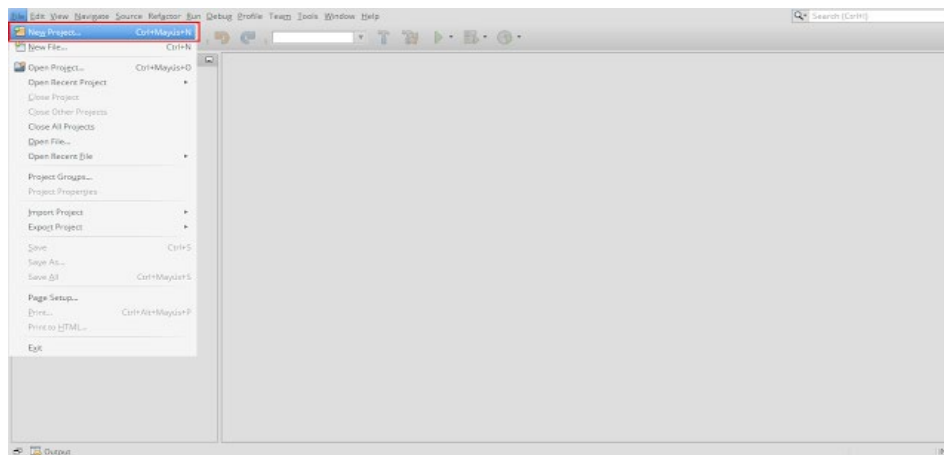
Vº Bº (de la autoridad)

Proyecto

Matemática con Netbeans 8.2

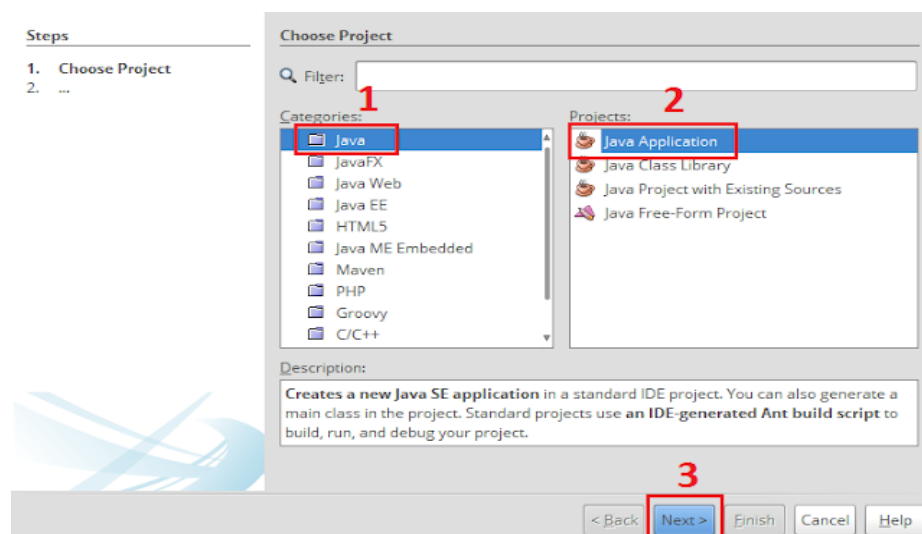
Nombre: Jugando con números enteros

En este paso crearemos el proyecto, para ello debes tener abierto NetBeans, más conocido como entorno de desarrollo (IDE). Comenzaremos dando click en File – New Project, como se muestra en la siguiente imagen.



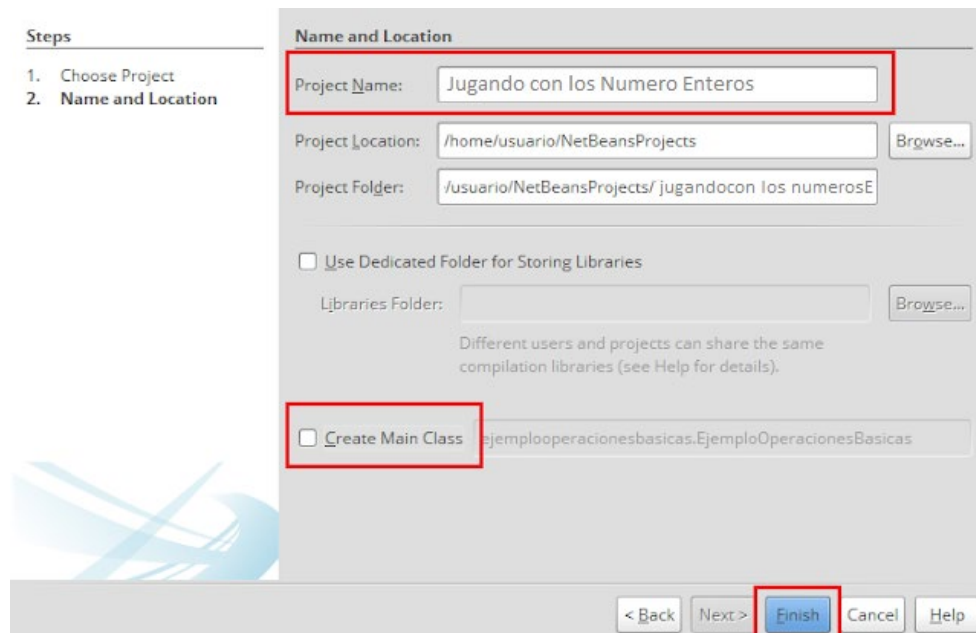
Fuente: Autoría propia

En la siguiente ventana se debe seleccionar: Categories – java y en Projects – Java Application, después dar click en Next, como se muestra en la siguiente imagen.



Fuente: Autoría propia

En la siguiente ventana se debe colocar un nombre al proyecto “Jugando con los números enteros”, sin espacios y sin tildes; también se debe desmarcar Create Main Class; por último, dar click en Finish, como se muestra en la siguiente imagen.

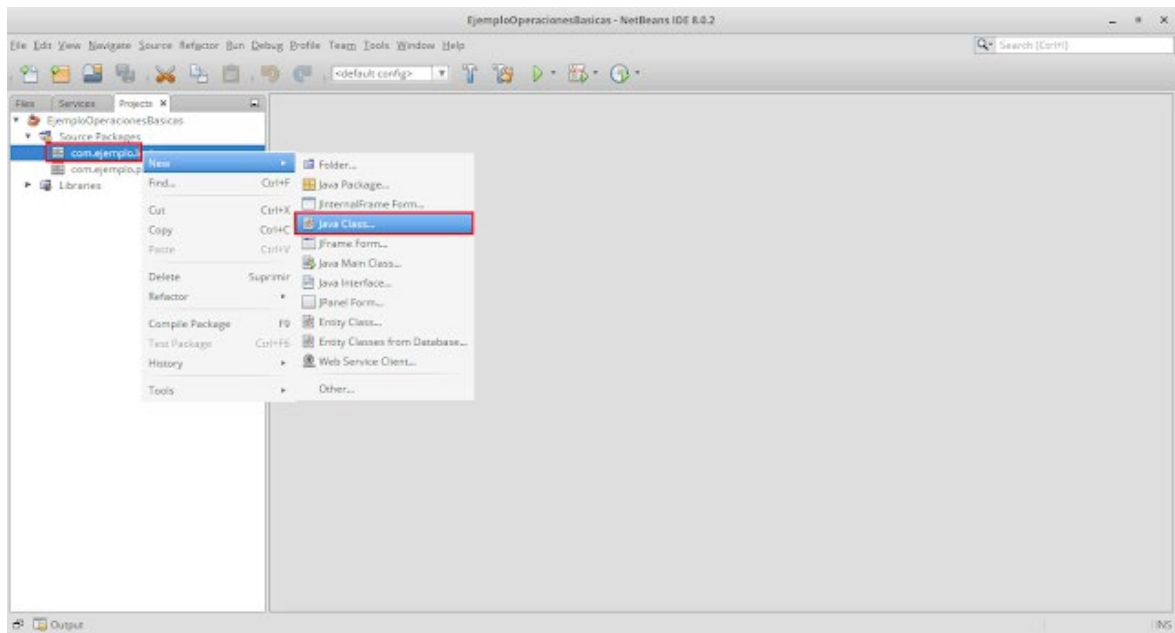


Fuente: Autoría propia

Crear una clase

En este paso crearemos la clase “OperacionesBasicas.java”, esta contendrá toda la lógica que nos ayudará a calcular las operaciones aritméticas, como lo son suma, resta, multiplicación y división. Por otra parte, crearemos cuatro métodos que recibirán como parámetros de entrada dos números de tipo double y nos retornará el resultado de dichas operaciones.

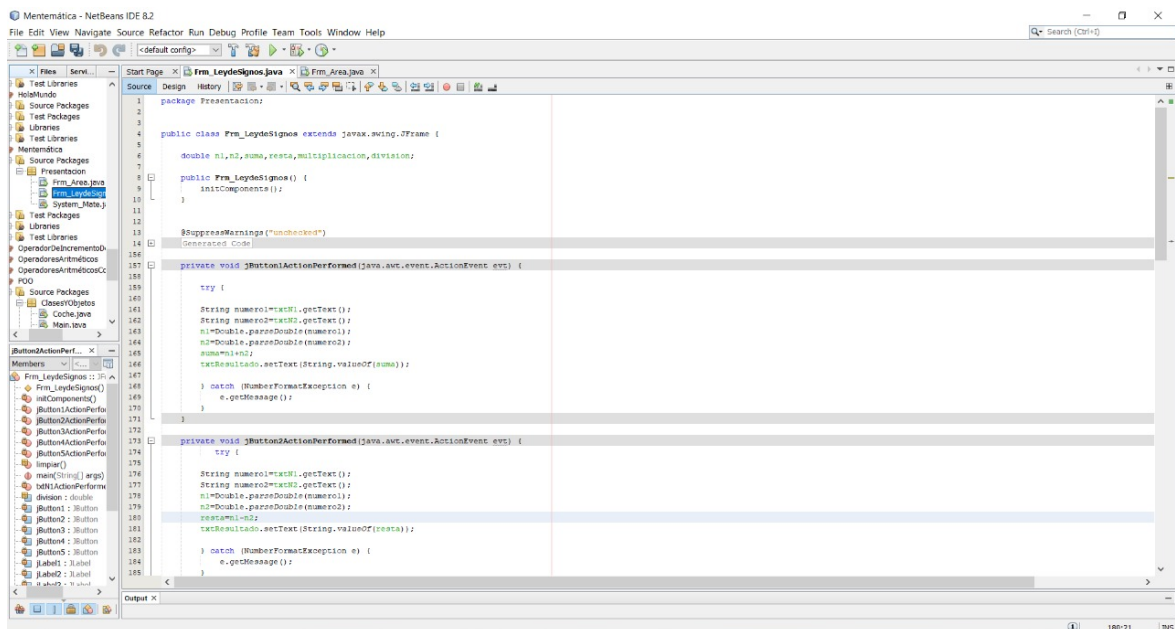
En la siguiente imagen se muestra la creación de una clase. Se debe dar click derecho sobre el paquete <com.ejemplo.logica>, después en New – JavaClass.



Fuente: Autoría propia

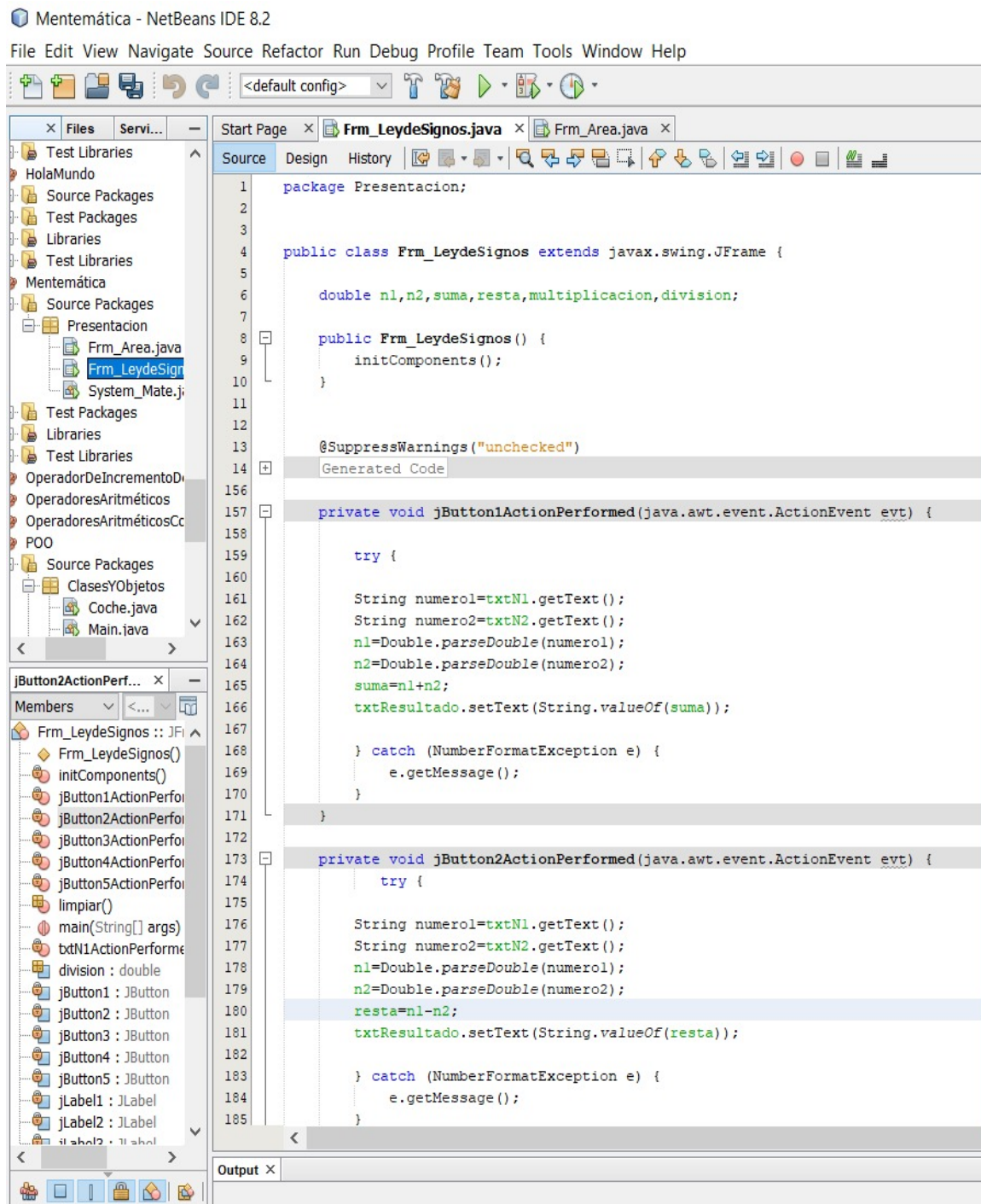
Luego de crear la clase, procederemos a nombrar la clase, sin espacios ni tildes, luego damos click en Finish.

Luego aparecerá la siguiente estructura:



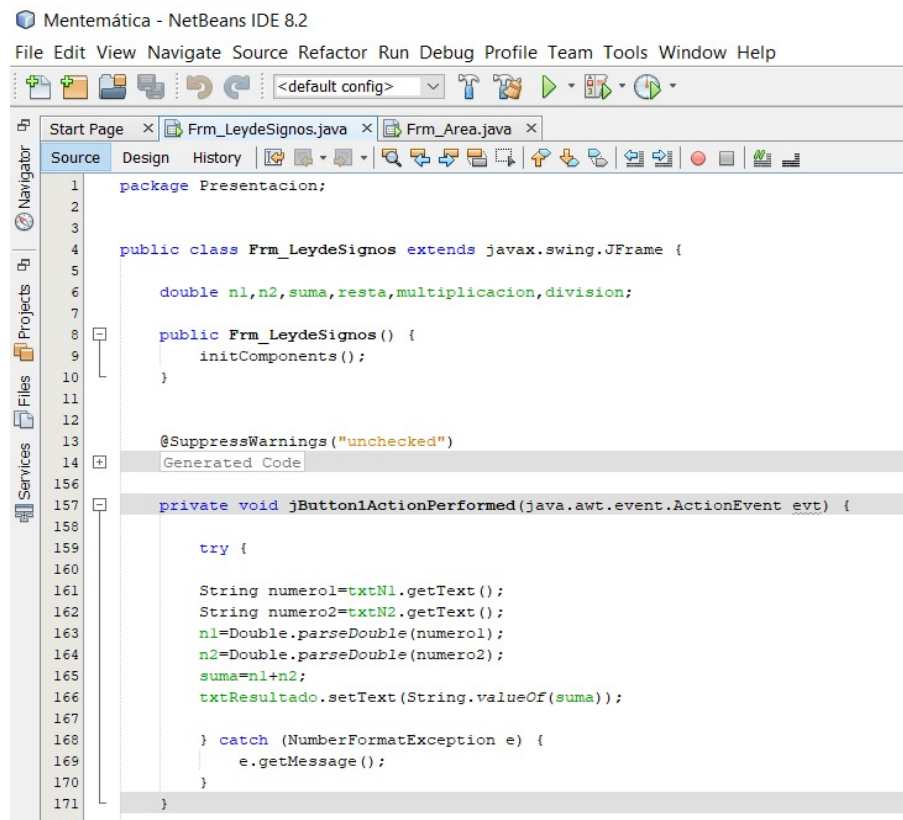
Fuente: Autoría propia

Ejecución del programa



Fuente: Autoría propia

Muestra por consola

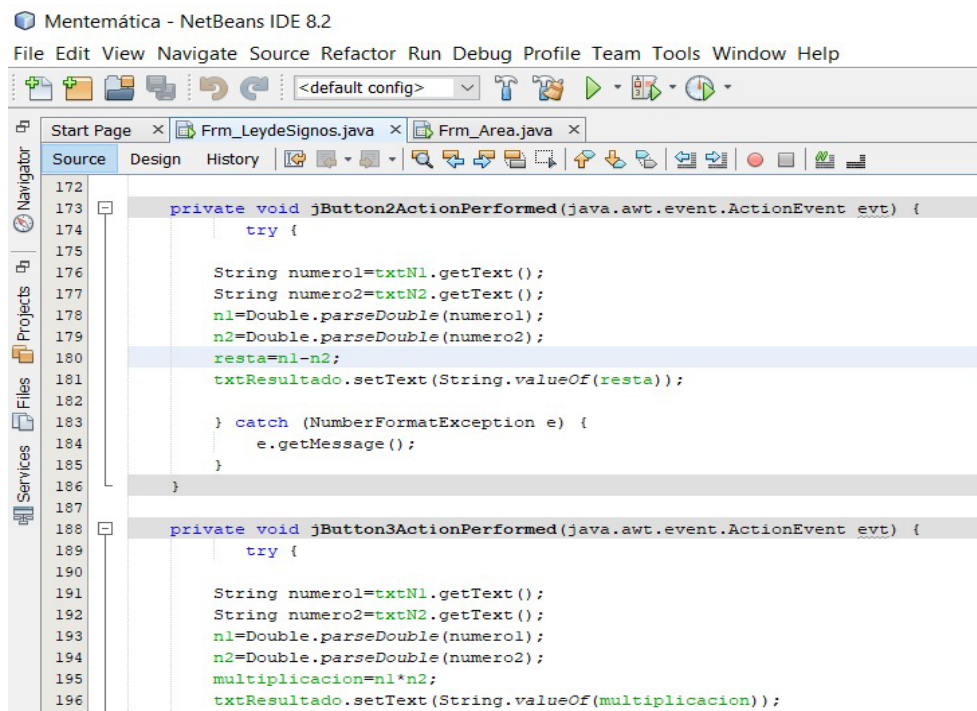


```

1 package Presentacion;
2
3
4 public class Frm_LeydeSignos extends javax.swing.JFrame {
5
6     double n1,n2,suma,resta,multiplicacion,division;
7
8     public Frm_LeydeSignos() {
9         initComponents();
10    }
11
12
13    @SuppressWarnings("unchecked")
14    Generated Code
15
16
17    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
18
19        try {
20
21            String numero1=txtN1.getText();
22            String numero2=txtN2.getText();
23            n1=Double.parseDouble(numero1);
24            n2=Double.parseDouble(numero2);
25            suma=n1+n2;
26            txtResultado.setText(String.valueOf(suma));
27
28        } catch (NumberFormatException e) {
29            e.getMessage();
30        }
31    }
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Fuente: Autoría propia



```

172
173
174 private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
175
176     try {
177
178         String numero1=txtN1.getText();
179         String numero2=txtN2.getText();
180         n1=Double.parseDouble(numero1);
181         n2=Double.parseDouble(numero2);
182         resta=n1-n2;
183         txtResultado.setText(String.valueOf(resta));
184
185     } catch (NumberFormatException e) {
186         e.getMessage();
187     }
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Fuente: Autoría propia

Mentemática - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Start Page x Frm_LeydeSignos.java x Frm_Area.java x

Source Design History

```

1 package Presentacion;
2
3
4 public class Frm_LeydeSignos extends javax.swing.JFrame {
5
6     double n1,n2,suma,resta,multiplicacion,division;
7
8     public Frm_LeydeSignos() {
9         initComponents();
10    }
11
12
13    @SuppressWarnings("unchecked")
14    Generated Code
15
16    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
17
18        try {
19
20            String numero1=txtN1.getText();
21            String numero2=txtN2.getText();
22            n1=Double.parseDouble(numero1);
23            n2=Double.parseDouble(numero2);
24            suma=n1+n2;
25            txtResultado.setText(String.valueOf(suma));
26
27        } catch (NumberFormatException e) {
28            e.getMessage();
29        }
30    }
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Fuente: Autoría propia

Mentemática - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Start Page x Frm_LeydeSignos.java x Frm_Area.java x

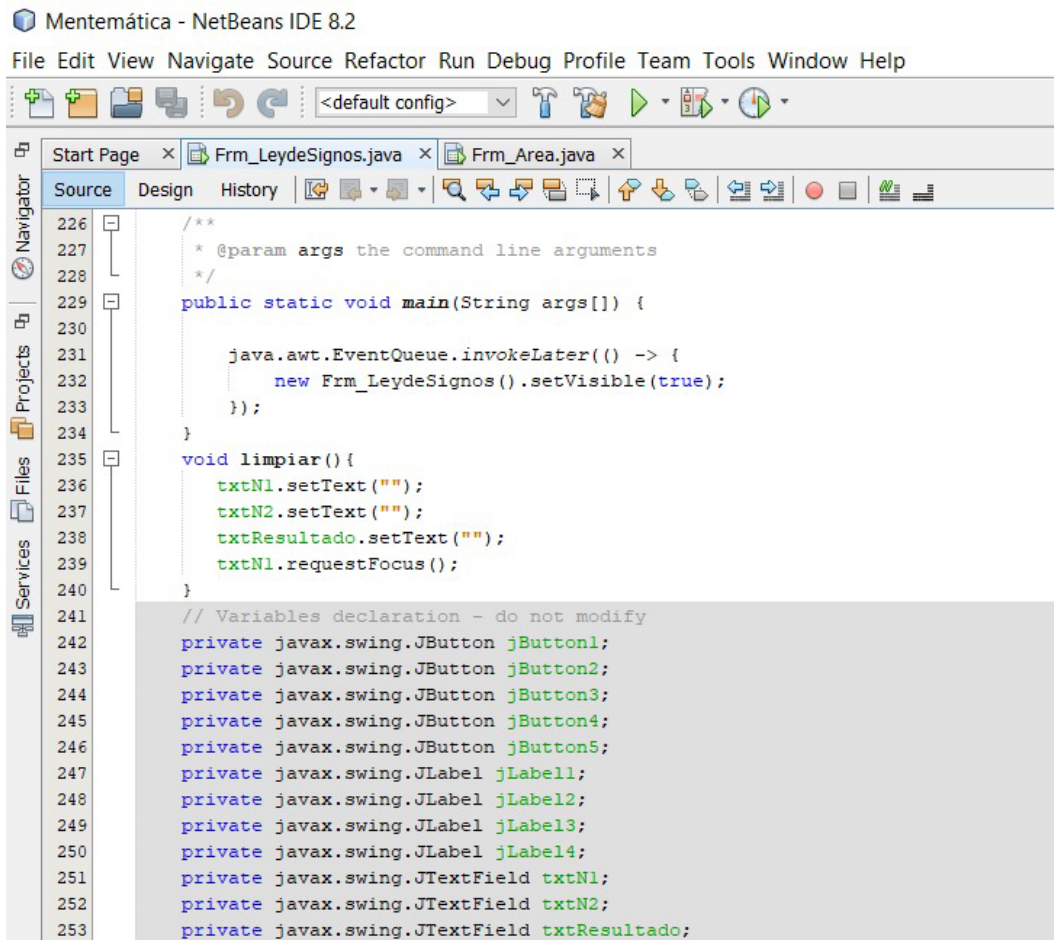
Source Design History

```

202
203 private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
204     try {
205
206         String numero1=txtN1.getText();
207         String numero2=txtN2.getText();
208         n1=Double.parseDouble(numero1);
209         n2=Double.parseDouble(numero2);
210         division=n1/n2;
211         txtResultado.setText(String.valueOf(division));
212
213     } catch (NumberFormatException e) {
214         e.getMessage();
215     }
216 }
217
218 private void jButton5ActionPerformed(java.awt.event.ActionEvent evt) {
219     limpiar();
220 }
221
222 private void txtN1ActionPerformed(java.awt.event.ActionEvent evt) {
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

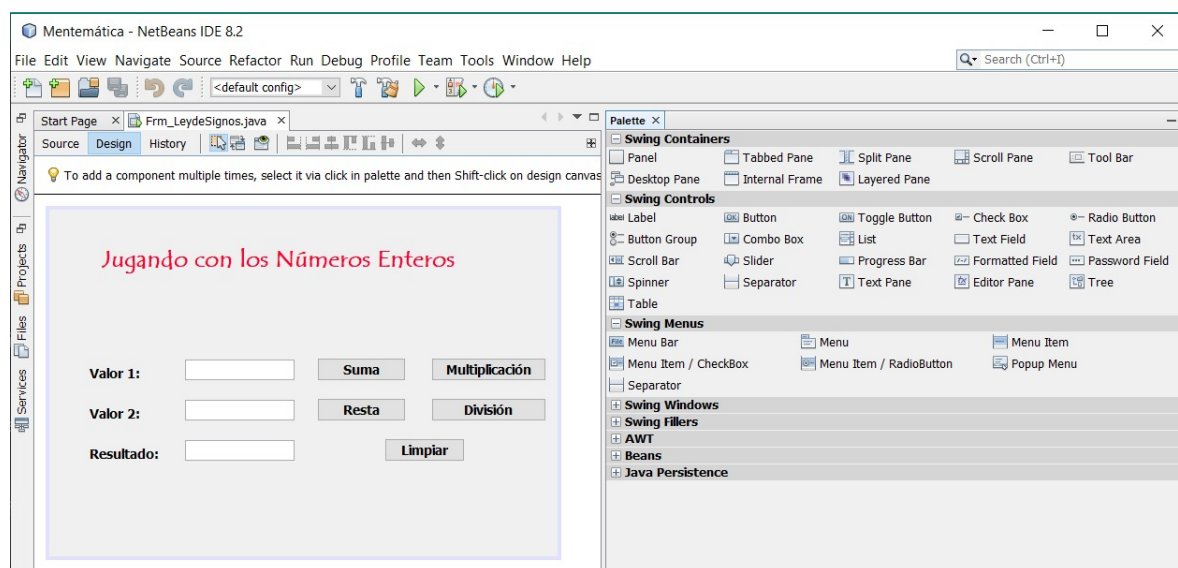
```

Fuente: Autoría propia



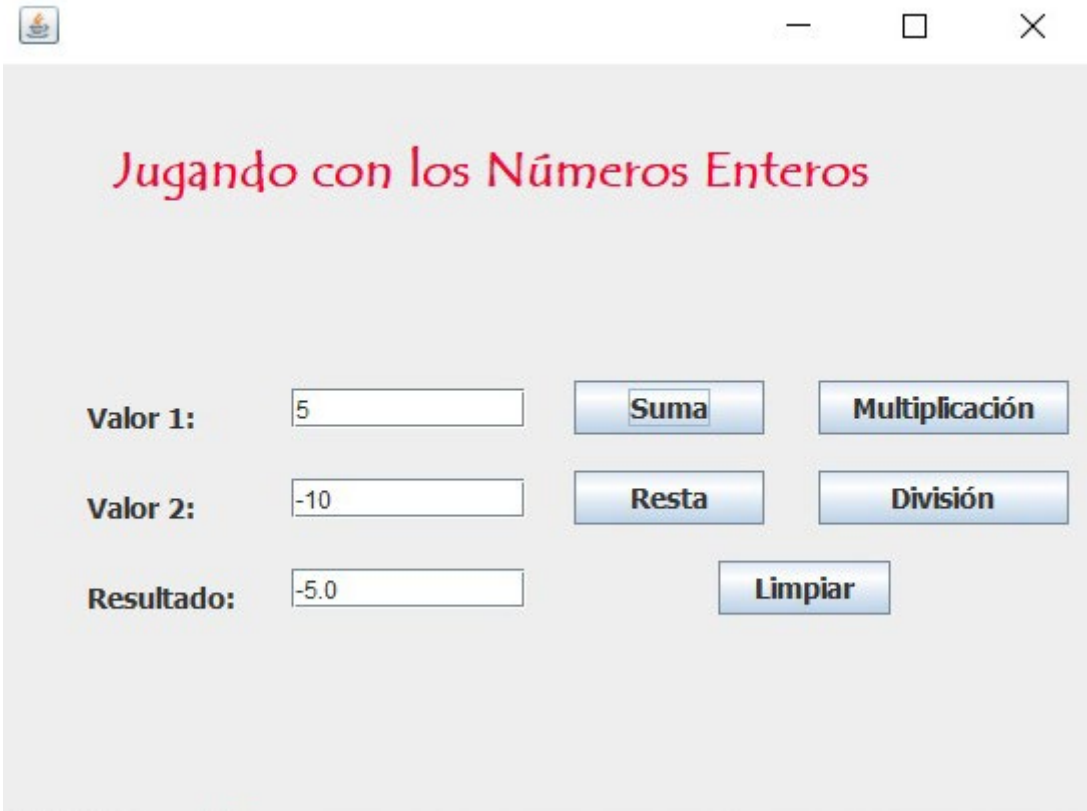
Fuente: Autoría propia

Diseño de interfaz



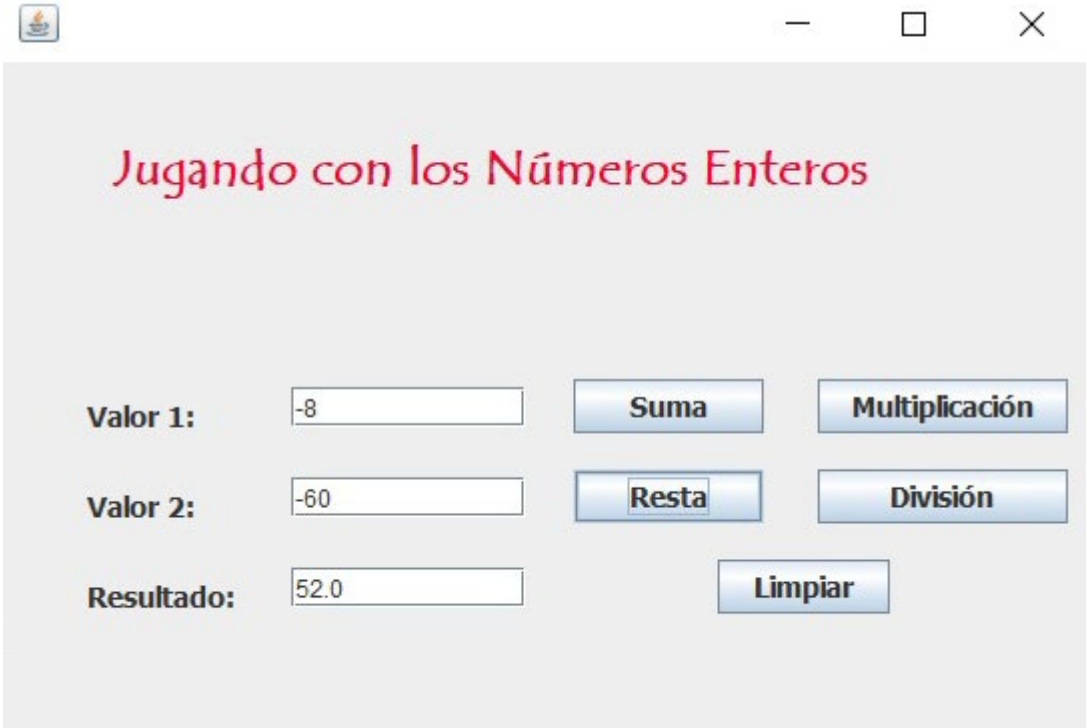
Fuente: Autoría propia

Ejecución de interfaz



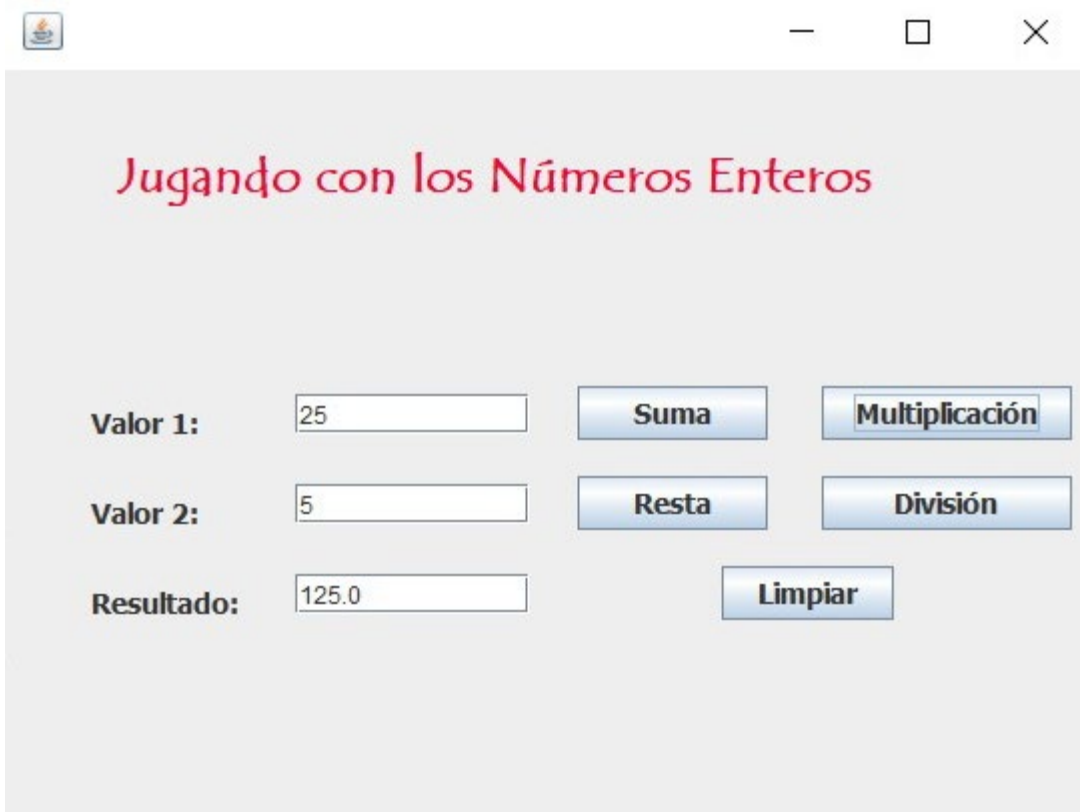
A Java Swing window titled "Jugando con los Números Enteros" with a standard Mac OS X title bar. The window contains a calculator interface. It has three input fields: "Valor 1:" with the value "5", "Valor 2:" with the value "-10", and "Resultado:" with the value "-5.0". To the right of these fields are five buttons: "Suma", "Multiplicación", "Resta", "División", and "Limpiar". The "Resta" button is highlighted with a blue border.

Fuente: Autoría propia



The same Java Swing window "Jugando con los Números Enteros" is shown in a different state. The input fields now contain "Valor 1:" with the value "-8", "Valor 2:" with the value "-60", and "Resultado:" with the value "52.0". The buttons "Suma", "Multiplicación", "Resta", "División", and "Limpiar" are still present. In this state, the "Resta" button is highlighted with a blue border.

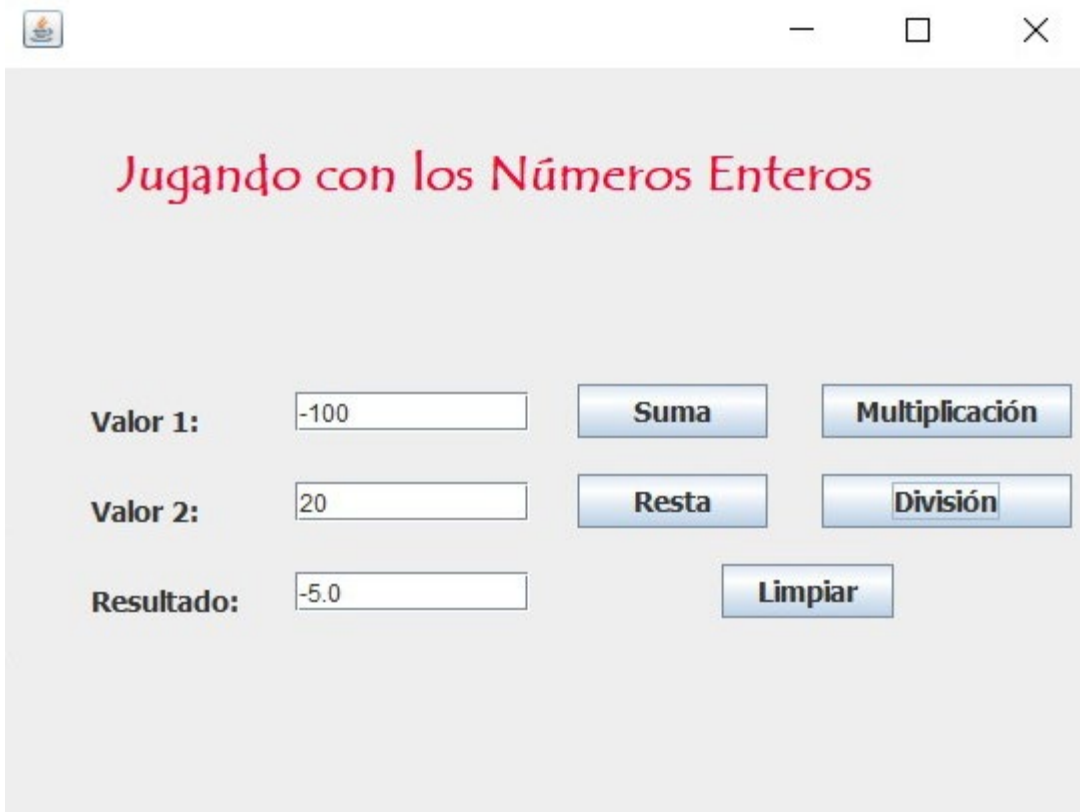
Fuente: Autoría propia



A Java Swing window titled "Jugando con los Números Enteros" with a standard Mac OS X title bar (red, yellow, and green buttons). The window has a light gray background. It contains three input fields and four buttons. The first row has "Valor 1:" followed by a text field containing "25", a "Suma" button, and a "Multiplicación" button. The second row has "Valor 2:" followed by a text field containing "5", a "Resta" button, and a "División" button. The third row has "Resultado:" followed by a text field containing "125.0" and a "Limpiar" button.

Valor 1:	25	Suma	Multiplicación
Valor 2:	5	Resta	División
Resultado:	125.0	Limpiar	

Fuente: Autoría propia



A Java Swing window titled "Jugando con los Números Enteros" with a standard Mac OS X title bar. The window has a light gray background. It contains three input fields and four buttons. The first row has "Valor 1:" followed by a text field containing "-100", a "Suma" button, and a "Multiplicación" button. The second row has "Valor 2:" followed by a text field containing "20", a "Resta" button, and a "División" button. The third row has "Resultado:" followed by a text field containing "-5.0" and a "Limpiar" button.

Valor 1:	-100	Suma	Multiplicación
Valor 2:	20	Resta	División
Resultado:	-5.0	Limpiar	

Fuente: Autoría propia

Síntesis

El reconocimiento de la importancia creciente del lenguaje de programación en el ámbito educativo y la demostración de integración y aceptación de distintos grupos con la necesidad de ampliar sus conocimientos, ateniéndose a soluciones y respetando ideologías aportadas por distintos individuos en virtud de la tolerancia y cooperación, compartiendo responsabilidad con el entorno y acatando orientaciones, ayudan al alumno a estar preparado para su futuro como profesional y poder adaptarse a las nuevas tecnologías que son cada más comunes en las nuevas generaciones. Así como el desarrollo de la necesidad de autoaprendizaje, como el desarrollo continuo de herramientas o instrumentos que faciliten la adaptación a la innovación tecnológica y organizativa, para la estabilidad laboral e inserción profesional.

Manteniendo el individuo una valoración de trabajo adecuada vinculada a la planificación, organización y desarrollo de actividades propias, donde la creatividad, iniciativa y responsabilidad mantenga el interés en el proceso educativo, derivándose en satisfacción profesional en relación con los resultados obtenidos, así como la valoración necesaria en la participación personal en la ejecución de los controles de calidad en la enseñanza y gestión de los conocimientos, siendo factores que facilitan el logro de resultados idóneos para la satisfacción de la población estudiantil.

La presente investigación monográfica está dividida en cuatro capítulos, en donde se detalla la ampliación del marco teórico respecto al lenguaje de programación, los cuales son muy utilizadas en diversas áreas tanto organizativas como académicas.

Apreciación crítica y sugerencias

La complejidad de la informática está vinculada al desarrollo y creación de los software, los cuales son un conglomerado de programas elaborados por personas de acuerdo a la utilidad e importancia para lo cual sea desarrollado, cuya trascendencia radica en el aprendizaje de lenguajes y programación partiendo de niveles básicos. De igual forma, es recomendable que la integración de cursos relacionados a la programación sean implementados en la malla curricular de los estudiantes no solo en los últimos grados de secundaria, puesto que actualmente los niños están muy familiarizados con las nuevas tecnologías, de manera que no va a ser tan complicado que los alumnos de distintos niveles de educación se adapten al entorno tecnológico, así como lo han estado haciendo desde el año 2020, en donde, a causa de la pandemia mundial, el sector educativo se vio obligado a tomar nuevas estrategias de enseñanza, optando por las clases remotas y accediendo a distintos programas tecnológicos que los alumnos han ido conociendo en estos últimos meses.

Referencias

- Álvarez, A., Ranilla, D., Unzueta, I. y González, M. (2015). *Lenguajes de programación*. Madrid, España. Recuperado de http://audax.zam.usal.es/web/mlperez/programacion_archivos/trabajos_2006_2007/tema2_AL_Alvarez_et_al.pdf
- Aristegui, O. (2010, 27 de marzo). Los casos de prueba en la prueba del software. *Revista digital Lámpasakos*. Recuperado de <https://dialnet.unirioja.es/servlet/articulo?codigo=3399441>
- Beristain, I., y Juárez, U. (2014). *Programación generativa en Java y herramientas de meta-programación*. D.F., México. Recuperado de https://res.cic.ipn.mx/2014_79/Programacion%20genertiva%20en%20Java%20y%20herramientas%20de%20meta-programacion.pdf
- Chavarri, E. (2004). *El ciclo hidrológico y la clasificación climática del Perú*. Lima, Perú. Recuperado de http://tarwi.lamolina.edu.pe/~echavarri/clase_ii_ciclo_hidrologico_clasif_clima_peru.pdf
- Durango, A., Arias, A. y Gracia, J. (2016). *Curso de programación con Java*. Bogotá, Colombia: Pirámide.
- Fernández, L. (2016). *Importancia y frecuencia del uso de Internet por parte de jóvenes estudiantes: percepción de una muestra del alumnado de la Universidad de La Laguna*. (Tesis de pregrado). Universidad de la Laguna, Madrid, España.
- Garrido, P. (2015). *Comenzando a programar con Java* (Tesis de pregrado). Universitat Miguel Hernández, Alicante, España.
- González, J. (2013). *Diseño y desarrollo de un depurador híbrido para java* (Tesis de pregrado) Universidad Politécnica de Valencia, España.
- Gortázar, F., Martínez, R. y Fresno, V. (2016). *Lenguajes de programación y procesadores*. Madrid, España: Ramón Areces.

- Hirsh, L. (2007). *Intérprete y entorno de desarrollo para el aprendizaje de lenguajes de programación estructurada* (Tesis de pregrado). Pontificia Universidad Católica del Perú, Lima, Perú.
- Inca, J. (2012). *Estudio del lenguaje de programación Haskell, ventajas y desventajas con respecto a otros lenguajes de programación* (Tesis de maestría). Pontificia Universidad Católica del Perú, Lima, Perú.
- Jiménez, N., Ampudia, P. y Sierro, D. (2010). *Implementación de un depurador declarativo para el lenguaje de bases de datos SQL*. Curso 2008-2009 (Tesis de maestría). Universidad Complutense de Madrid, España.
<https://eprints.ucm.es/id/eprint/9830/1/memoria.pdf>
- Joyanes, L. (2008). *Fundamentos de programación. algoritmos, estructura de datos y objetos*. Madrid, España: Mc Graw Hill.
- Lázcares, R. y Molina, A. (2015). *Lenguajes de programación COBOL y PL/I, historia y características principales*. Universidad de Costa Rica, San José, Costa Rica.
- Lorente, K., Cervantes, G. y Salgado, A. (2020, 8 de enero). Aprendizaje de estructuras de datos mediante algoritmos usando pseudocódigos. *Revista Electrónica Formación y Calidad Educativa*. Recuperado de <http://refcale.uileam.edu.ec/index.php/refcale/article/view/3181/1983>
- Luján, S. (2002). *Programación de aplicaciones web: historia, principios básicos y clientes web*. Madrid, España: Club Universitario.
- Mancilla, A., Ebratt, R., Capacho, J. (2016). *Diseño y construcción de algoritmos*. Barranquilla, Colombia: Ediciones de la U.
- Manrique, E. (2020, 28 de enero). Machine Learning: análisis de lenguajes de programación y herramientas para desarrollo. *Revista Ibérica de Sistemas y*

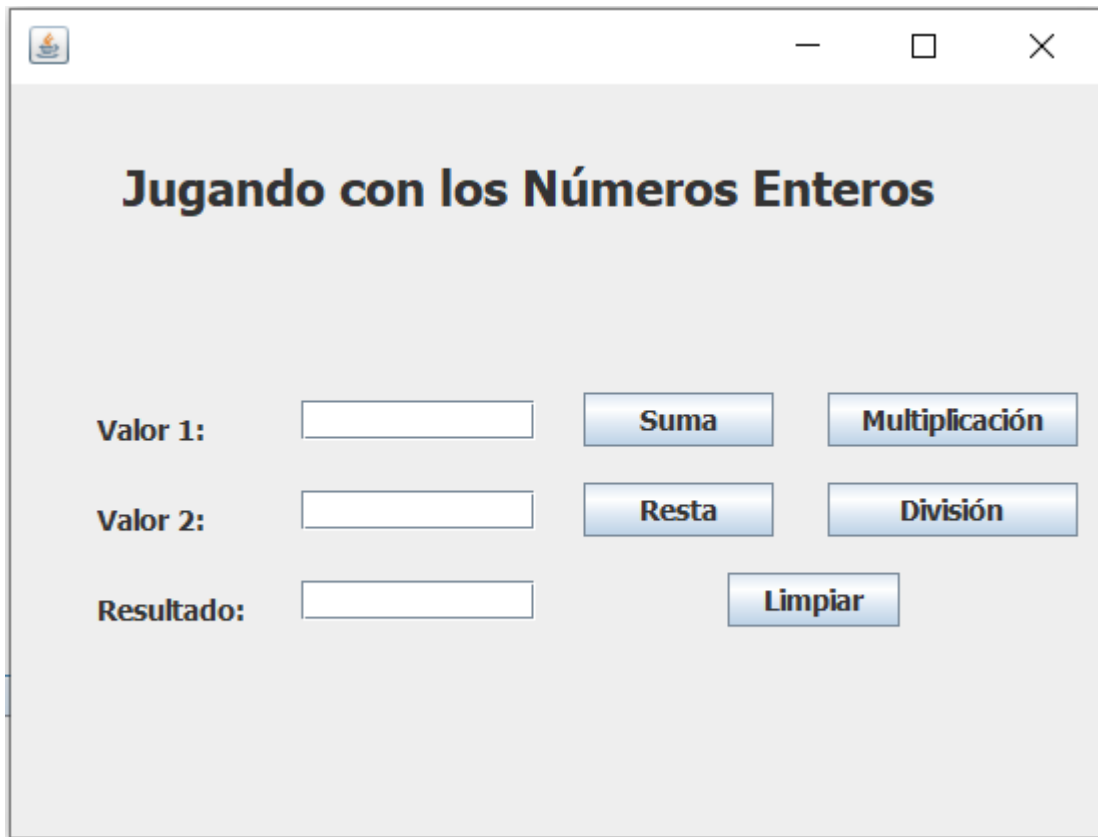
- Tecnologías de Información*. Recuperado de <https://www.proquest.com/openview/c7e24c997199215aa26a39107dd2fe98/1?pq-origsite=gscholar&cbl=1006393>
- Márquez, R. (2017). *El principio FIRST, o cómo aumentar la calidad de nuestros tests unitarios*. Paradigma. Monterrey, México. Recuperado de <https://www.paradigmadigital.com/dev/principio-first-aumentar-la-calidad-tests-unitarios/>
- Merino, E. (2015). *Diseño de un simulador de compilador para plataforma Moodle e implementación de un laboratorio virtual para la enseñanza de programación* (Tesis de maestría). Pontificia Universidad Católica del Ecuador, Quito, Ecuador.
- Moreno, L. (2016). *Java como lenguaje universal de programación*. XIKUA Boletín Científico de la Escuela Superior de Tlahuelilpan, D.F., México. Recuperado de <https://repository.uaeh.edu.mx/revistas/index.php/xikua/article/view/332>
- Rojas, J. (2018). *Compresión de tweets usando algoritmo de Hoffman con modelos predefinidos* (Tesis de pregrado). Universidad Técnica Federico Santa María, Santiago, Chile.
- Rodríguez, J. (2016). *Introducción a la programación. Teoría y práctica*. Alicante, España: Club Universitario.
- Schmidt, M., Parra, G. (2003). *Sobre la construcción de compiladores en Java*. Red de Universidades con Carreras Informáticas. Buenos Aires, Argentina: Recuperado de <http://sedici.unlp.edu.ar/handle/10915/21559>
- Sierra, K., y Bates, B. (2005). *Primero Java*. Nueva York, E.U.: O'Reilly Media.
- Trejos, O. (2013, 18 de mayo). Relaciones de aprendizaje significativo entre dos paradigmas de programación a partir de dos lenguajes de programación. *Revista Tecnura*, Recuperado de <https://revistas.udistrital.edu.co/index.php/Tecnura/article/view/7025/8703>

Viera, N. (2019). *Implementación de un generador de casos de prueba para procedimientos en lenguaje java* (Tesis de pregrado). Universidad Nacional de Piura, Perú.

Apéndice

Apéndice A: Programa matemático en Java

Apéndice A: Programa matemático en Java



The image shows a Java Swing window with a light gray background. The title bar at the top contains a small icon on the left and standard window controls (minimize, maximize, close) on the right. The main content area has the title "Jugando con los Números Enteros" in a bold, black, sans-serif font. Below the title, there are three rows of input fields and buttons. The first row has the label "Valor 1:" followed by a text input field, a blue button labeled "Suma", and another blue button labeled "Multiplicación". The second row has the label "Valor 2:" followed by a text input field, a blue button labeled "Resta", and another blue button labeled "División". The third row has the label "Resultado:" followed by a text input field and a blue button labeled "Limpiar". All buttons have a blue gradient and a slight 3D effect.

Fuente: Autoría propia.